



Geometrical Transformations





Map of the course

- **Basic Transformations of the Plane**
- **Homogeneous Coordinates**
- **Composition of 2D Transformations**
- **Basic Transformations of the Space**
- **Composition of 3D Transformations**
- **Hierarchical Modeling**



Basic Transformations of the Plane

■ **Problem:** *How to move an object*

- We know how to move objects in real life
- How can we move them on a computer?
- Goal: move the house from “here” to “there”
- Need: a way of quantifying “here” and “there”
- Solution: a coordinate system
 - Expresses space numerically
 - Provides metric (can describe the distance between one point and another)

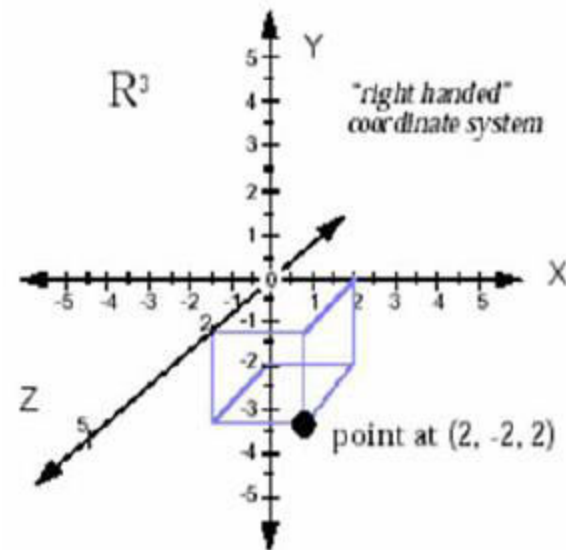
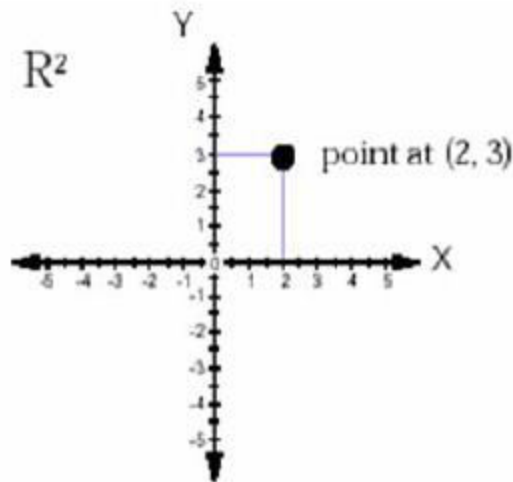
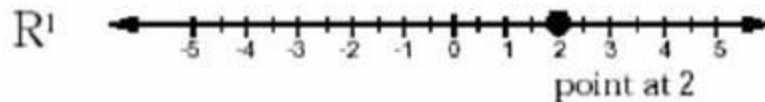




Basic Transformations of the Plane

■ **Problem:** *How to move an object*

- Examples: one, two and three dimensional real coordinate spaces.

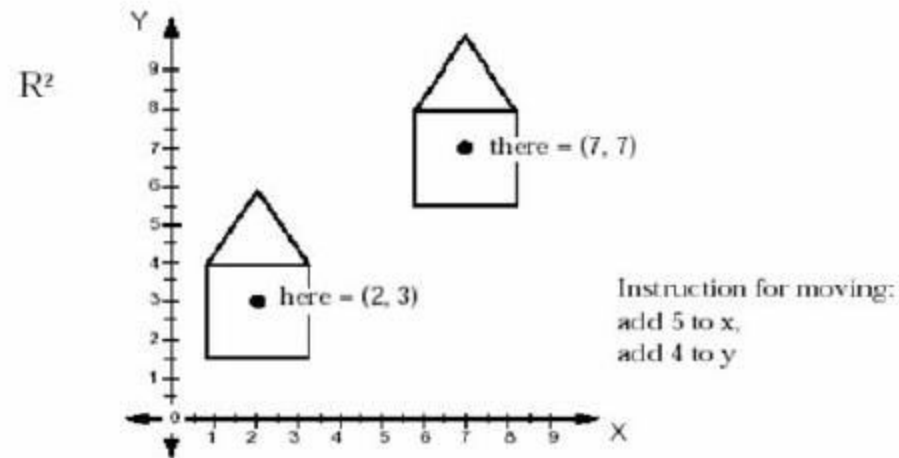
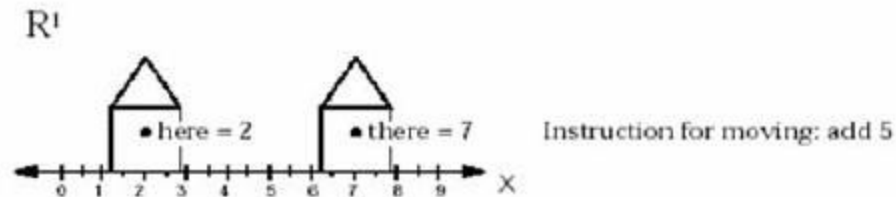




Basic Transformations of the Plane

■ **Problem:** *How to move an object*

- Now we can give quantitative instructions for moving the house:





Basic Transformations of the Plane

- **Problem:** *How to move an object*
 - Vectors, and the operations of addition, scalar multiplication, and dot product will be used to:
 - translate (move)
 - rotate
 - scale
 - ...
 - Objects made on the computer
 - To move polygons: just **move** vertices (vectors) and then redraw lines between them.



Basic Transformations of the Plane

■ Translation:

- Component-wise addition of vectors : $P' = P + T$
where :

$$P = \begin{bmatrix} x \\ y \end{bmatrix}, \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix}, \quad T = \begin{bmatrix} dx \\ dy \end{bmatrix}$$

and $x' = x + dx$

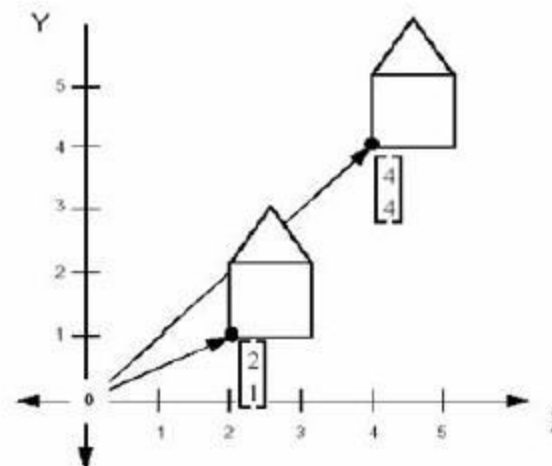
$$y' = y + dy$$

$$dx = 2$$

$$dy = 3$$

- **Rigid-body transformation:**

- Preserves lengths
- Preserves angles



Note: House shifts position relative to origin

Basic Transformations of the Plane

Scaling:

- Component-wise scalar multiplication of vectors $P' = SP$

where :

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

and

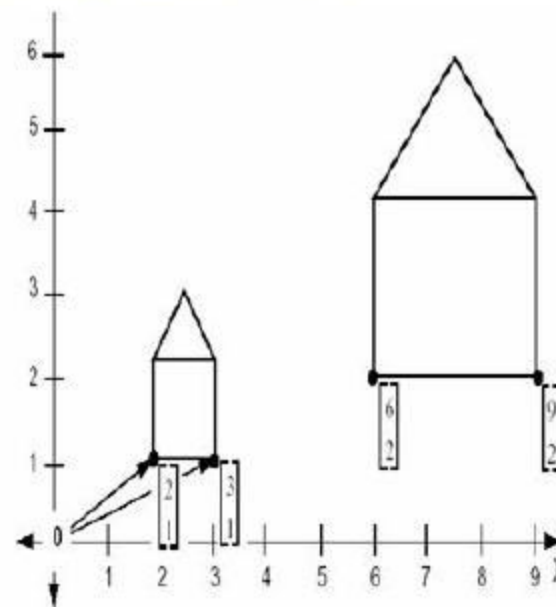
$$x' = s_x x$$

$$y' = s_y y$$

$$s_x = 3$$

$$s_y = 2$$

- Does not preserve lengths.
- Does not preserve angles ($s_x \neq s_y$)
- S_x, S_y are positive values.
- Scaling is not rigid body transformation.



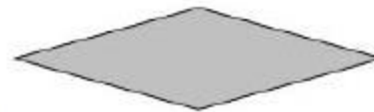
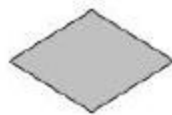
Note: House shifts position relative to origin



Basic Transformations of the Plane

■ **Scaling:**

- Does not preserve lengths
- Does not preserve angles ($s_x \neq s_y$)



Non-uniformly scaled

- **Affine transformation:**
 - **Preserving Parallelism**
 - **But not length and angles**



Basic Transformations of the Plane

■ **Rotation:**

- A two dimensional rotation of an object is obtained by repositioning the object along a *circular path* in the *xy plane*.
- In this case, we are rotating the object about a rotation axis that is perpendicular to the *xy plane*.
- To simplify the explanation of the basic method, we first determine the transformation equations for rotation of a point **P** when the pivot point is at the *coordinate origin*.
 - **The pivot point:** the point about which the object is to be rotated.



Basic Transformations of the Plane

■ Rotation:

- Rotation of vectors through an angle θ

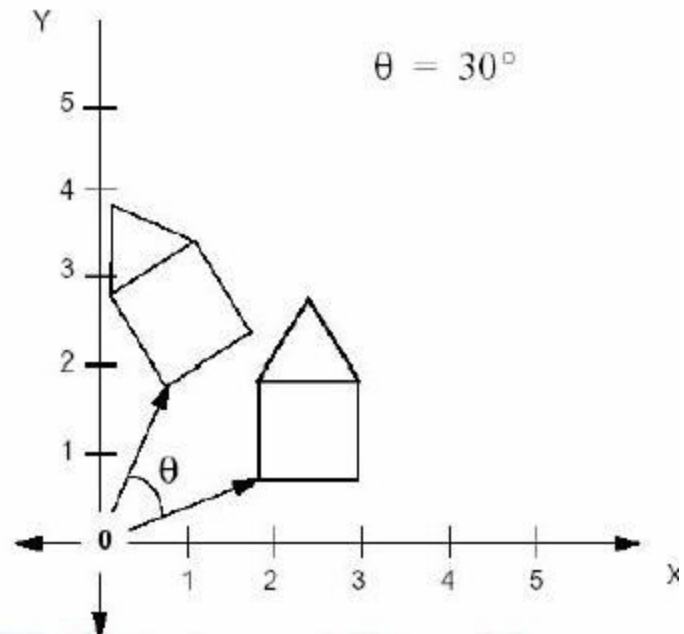
$$P' = R_{\theta}P \quad \text{where } P = \begin{bmatrix} x \\ y \end{bmatrix}, \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

and

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$\mathbf{R}_{\theta} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$



- *Rotation is rigid body transformation*
 - *Preserves lengths and angles*

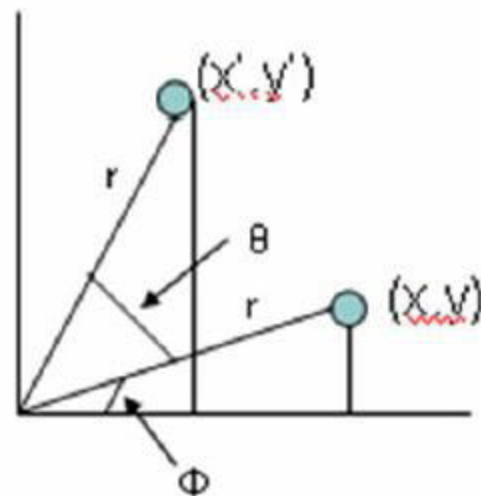
Note: house shifts position relative to the origin



Basic Transformations of the Plane

■ Mathematic Recall:

- $x = r \cdot \cos \Phi,$
- $y = r \cdot \sin \Phi$
- $x' = r \cdot \cos(\theta + \Phi) = r \cdot \cos \Phi \cdot \cos \theta - r \cdot \sin \Phi \cdot \sin \theta,$
- $y' = r \cdot \sin(\theta + \Phi) = r \cdot \cos \Phi \cdot \sin \theta + r \cdot \sin \Phi \cdot \cos \theta,$
- $x' = x \cdot \cos \theta - y \cdot \sin \theta,$
- $y' = x \cdot \sin \theta + y \cdot \cos \theta,$





Basic Transformations of the Plane

■ Summary:

- Translation, scaling and rotation are expressed as:
 - Translation: $P' = P + T$
 - Scale: $P' = SP$
 - Rotation: $P' = RP$
- Translation and rotation preserves angles and length
 - Using special matrix: **orthogonal matrix**
 - **Rigid-body transformation.**



Map of the course

✓ Basic Transformations of the Plane

✗ Homogeneous Coordinates

- Composition of 2D Transformations
- Basic Transformations of the Space
- Composition of 3D Transformations
- Hierarchical Modeling



Homogenous Coordinates

- Many graphics applications involve sequences of geometric transformations.
 - An *animation* might require an object to be translated and rotated at each increment of the motion.
 - *Viewing transformation* involve sequences of translations and rotations to take use from the original scene specification to the display on the output device.
- A more efficient approach, however, is to *combine the transformations* so the that the final coordinate positions are obtained directly from the initial coordinates, *without calculating intermediate coordinate values*.



Homogenous Coordinates

- Multiplicative and transitional terms for a two-dimensional geometric transformation can be **combined into a single matrix if we expand the representation to 3 by 3 matrices.**
- Then, we can use the **third column** of a transformation matrix for the translation terms, **and all transformation equations can be expressed as matrix multiplications.**
- But, to do so, we also need to expand the matrix representation for a two dimensional coordinate position to a three element column matrix.
- A standard technique for accomplishing this is to expand each two-dimensional coordinate-position representation (x, y) to a three element representation (x_h, y_h, h) , called **homogeneous coordinates**, where **the homogeneous parameter h is a nonzero value.**

Homogenous Coordinates

- Homogeneous coordinate representation allows transformation of *n-dimensional* vectors in *(n+1) dimensional* space.
- The transformed *n-dimensional* results are obtained by *projecting* the transformed *(n+1)-dimensional point* back into the *n-dimensional* space of interest.

$$\begin{array}{ccc} \begin{bmatrix} x \\ y \end{bmatrix} & \longrightarrow & \begin{bmatrix} x \\ y \\ H \end{bmatrix} \\ \text{Ordinary} & & \text{Homogeneous} \\ \text{coordinate} & & \text{coordinate} \end{array}$$



Homogenous Coordinates

- In homogeneous coordinates we add a **third coordinate** to our point representation, so each point is represented by a triplet (x, y, h) .
- If homogenize the point (divide by h), we get a point of the form $(x, y, 1)$.
- So, the homogenized points form the plane defined by $h=1$.
- **The cartesian coordinate x - y plane can be represented by the $h = 1$ plane in a homogeneous coordinate representation.**



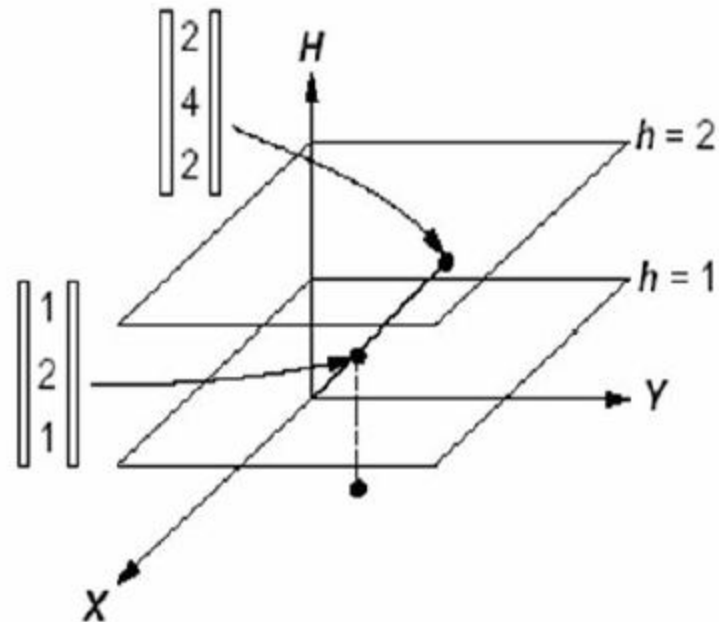
Homogenous Coordinates

- To obtain the Cartesian coordinate which is represented by a homogeneous coordinate divide each entry by h .

$$\begin{bmatrix} 2 \\ 4 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Homogeneous

Cartesian





Homogenous Coordinate Transformations

- For points written in homogeneous coordinates
- Translation, scaling and rotation are expressed homogeneously as:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$T(dx, dy) = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Basic Transformations of the Plane

- **Inverse Transformations:**

- **Translation:**

- We obtain the inverse matrix by *negating the translation distances*.

$$\mathbf{T}^{-1} = \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- This produces a translation in the opposite direction.
 - The product of a translation matrix and its inverse produce the **identity matrix**



Basic Transformations of the Plane

- **Inverse Transformations:**

- **Rotation:**

- Accomplished by replacing the rotation angle by its negative.

$$\mathbf{R}^{-1} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Negative values for rotation angles generate rotations in a **clockwise direction**.
 - The identity matrix is produced when any rotation matrix is multiplied by its inverse.



Basic Transformations of the Plane

■ Inverse Transformations:

■ Scaling:

- We form the inverse matrix for any scaling transformation by replacing the scaling parameters with their reciprocals.

$$S^{-1} = \begin{bmatrix} \frac{1}{S_x} & 0 & 0 \\ 0 & \frac{1}{S_y} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- The inverse matrix generates an opposite scaling transformation.
- The identity matrix is produced when any scaling matrix is multiplied by its inverse.



Map of the course

- ✓ Basic Transformations of the Plane
- ✓ Homogeneous Coordinates
- ✍ **Composition of 2D Transformations**
 - **Basic Transformations of the Space**
 - **Composition of 3D Transformations**
 - **Hierarchical Modeling**

Composition of 2D Transformations -1

■ Translation Composition:

$$P' = T(\Delta x_1, \Delta y_1)P$$

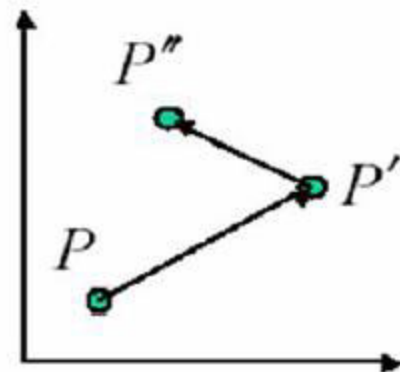
$$P'' = T(\Delta x_2, \Delta y_2)P'$$

$$= T(\Delta x_2, \Delta y_2)T(\Delta x_1, \Delta y_1)P$$

$$= \begin{bmatrix} 1 & 0 & \Delta x_2 \\ 0 & 1 & \Delta y_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & \Delta x_1 \\ 0 & 1 & \Delta y_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & \Delta x_1 + \Delta x_2 \\ 0 & 1 & \Delta y_1 + \Delta y_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= T(\Delta x_1 + \Delta x_2, \Delta y_1 + \Delta y_2)P$$



Successive translations are additive



Composition of 2D Transformations -2

■ Scale Composition:

$$\begin{aligned}P'' &= S(\Delta x_2, \Delta y_2)S(\Delta x_1, \Delta y_1)P \\&= \begin{bmatrix} \Delta x_2 & 0 & 0 \\ 0 & \Delta y_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x_1 & 0 & 0 \\ 0 & \Delta y_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \\&= \begin{bmatrix} \Delta x_1 \Delta x_2 & 0 & 0 \\ 0 & \Delta y_1 \Delta y_2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \\&= S(\Delta x_1 \Delta x_2, \Delta y_1 \Delta y_2)P\end{aligned}$$

Successive scalings are multiplicative



Composition of 2D Transformations -3

■ Rotation Composition:

$$P'' = R(\phi)R(\theta)P$$

$$= \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \phi \cos \theta - \sin \phi \sin \theta & -\cos \phi \sin \theta - \sin \phi \cos \theta & 0 \\ \sin \phi \cos \theta + \cos \phi \sin \theta & -\sin \phi \sin \theta + \cos \phi \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos(\phi + \theta) & -\sin(\phi + \theta) & 0 \\ \sin(\phi + \theta) & \cos(\phi + \theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

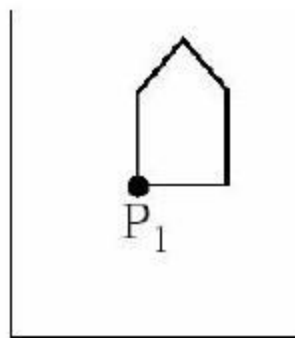
$$= R(\phi + \theta)P$$

Successive rotations are additive

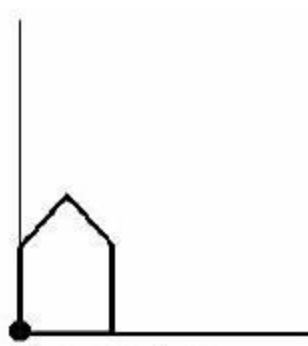


Composition of 2D Transformations -4

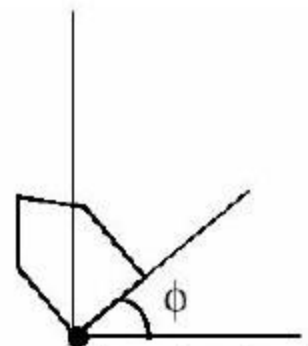
- **Rotates about point P_1 ?**
 - Translate P_1 to origin
 - Rotate
 - Translate origin back to P_1



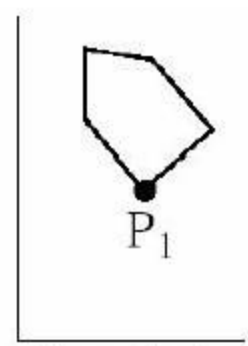
House at P_1



Translation
of P_1 to Origin



Rotation by ϕ



Translation
back to P_1



Composition of 2D Transformations -4

- $R(\Phi)$ rotates about the origin; to rotate **about point P_1** ?
 - Translate P_1 to origin
 - Rotate
 - Translate origin back to P_1

$$\begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix} =$$

Translation back to P_1 Rotation by ϕ Translation to origin



Composition of 2D Transformations -4

- $R(\Phi)$ rotates about the origin; to rotate about point P_1 ?

$$\begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \phi & -\sin \phi & 0 \\ \sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix} =$$

Translation back to P_1 Rotation by ϕ Translation to origin

$$\begin{bmatrix} \cos \phi & -\sin \phi & x_1(1-\cos \phi) + y_1 \sin \phi \\ \sin \phi & \cos \phi & y_1(1-\cos \phi) - x_1 \sin \phi \\ 0 & 0 & 1 \end{bmatrix}$$



Composition of 2D Transformations -5

scaling an object with respect to a specified fixed point position

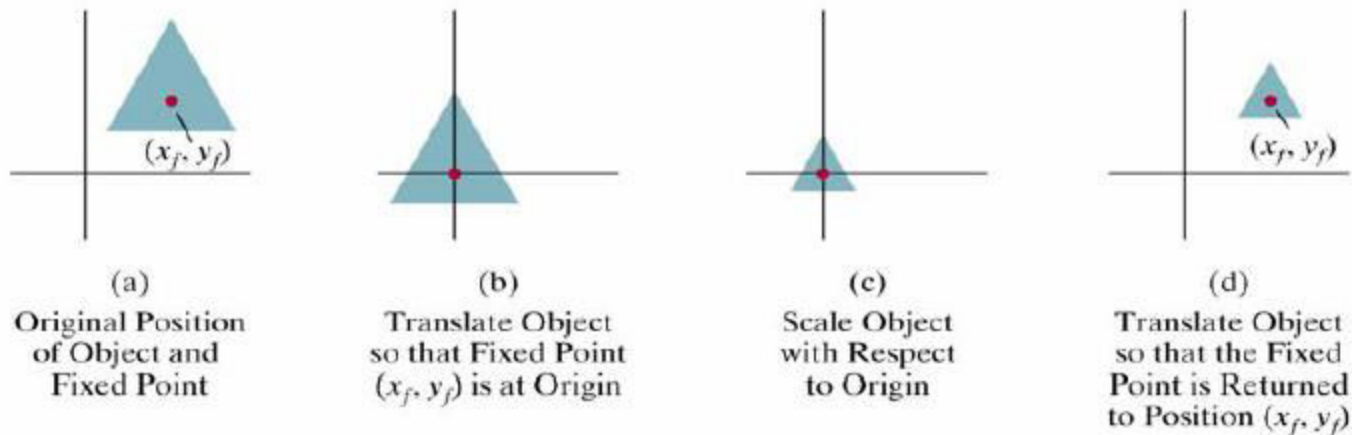


Figure 5-10

A transformation sequence for scaling an object with respect to a specified fixed position using the scaling matrix $S(s_x, s_y)$ of transformation 5-21.

A transformation sequence for scaling an object with respect to a specified fixed point position using the scaling matrix $S(s_x, s_y)$ of transformation



Composition of 2D Transformations -5

- The sequence is:
 - Translate the object so that fixed point coincides with the coordinate origin.
 - Scale the object with respect to the coordinate origin.
 - Use the inverse of the translation in step (1) to return the object to its original position

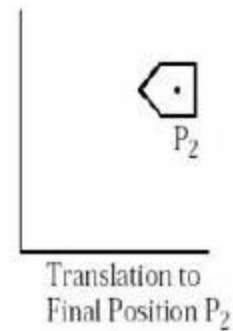
$$\begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & x_f(1-s_x) \\ 0 & s_y & y_f(1-s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

$$T(x_f, y_f).S(x_f, y_f).T(-x_f, -y_f) = S(x_f, y_f, s_x, s_y)$$



Composition of 2D Transformations

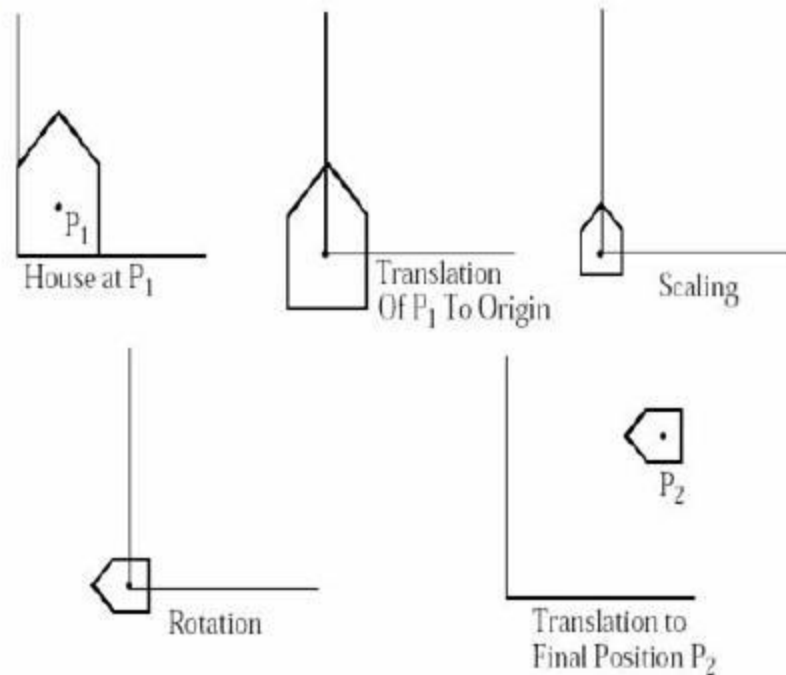
- **To scale, rotate and translate an arbitrary symbol about center P_1 , then place at P_2 :**
- Scale, rotate about point P_1
 - Translate P_1 to origin
 - Scale
 - Rotate
- Translate origin to P_2





Composition of 2D Transformations

- **To scale, rotate and translate an arbitrary symbol about center P_1 , then place at P_2 :**
- Scale, rotate about point P_1
 - Translate P_1 to origin
 - Scale
 - Rotate
 - Translate origin to P_2





Composition of 2D Transformations

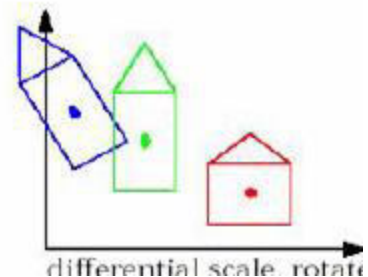
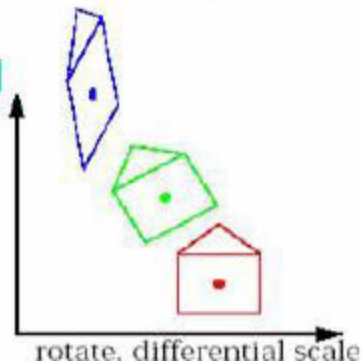
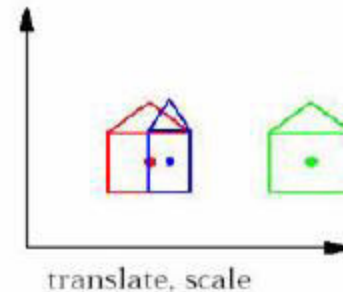
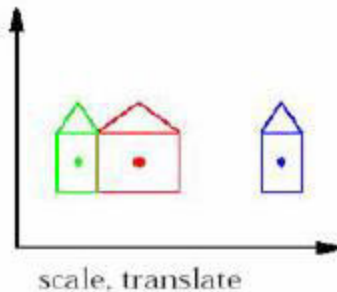
■ **Non-Commutative Combinations of Transformations in 2D:**

- non-uniform scale, rotate– translate, scale
- rotate, translate
- Non-commutative

■ **red is the original**

■ **green is the transitional**

■ **blue is the final figure**

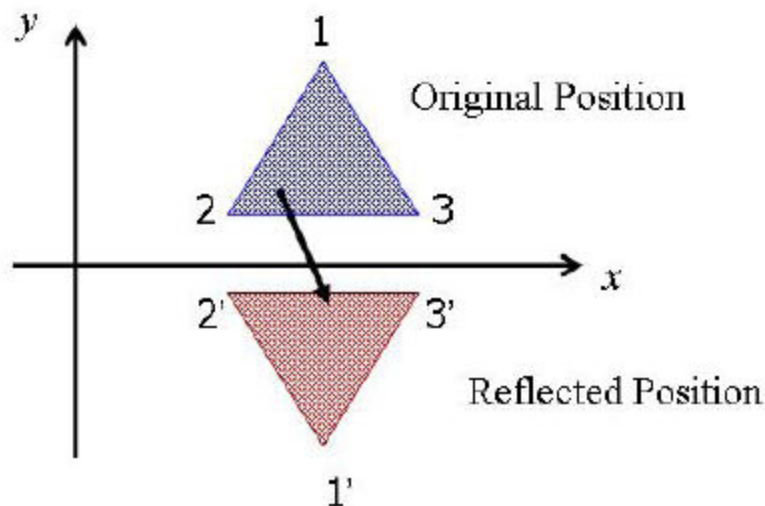




Other two-dimensional transformation -1

■ Reflection:

- A transformation that produces a mirror image of an object.
- For 2D transformation, reflection is generated relative to an **axis of reflection** by rotating the object **180°**.



$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

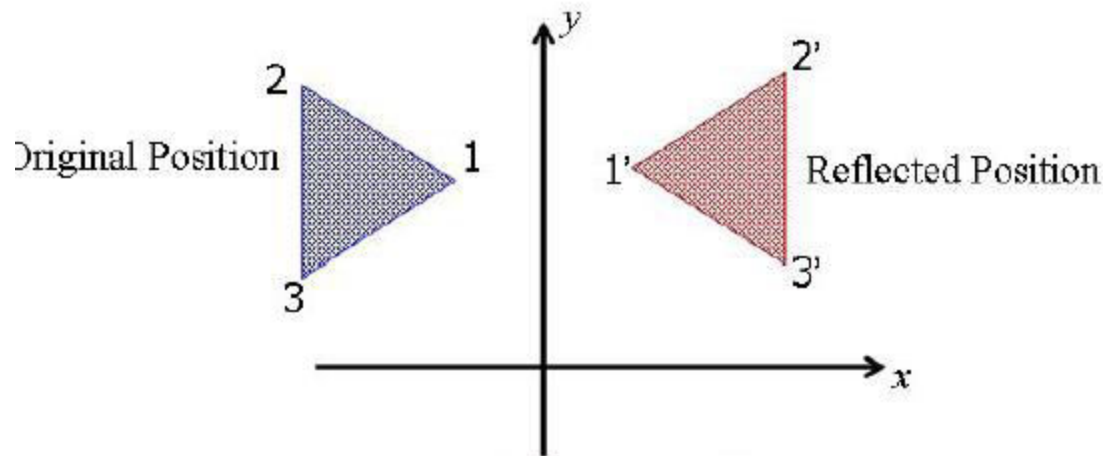
Transformation Matrix

Reflection of an object about the x axis

Other two-dimensional transformation -2

■ Reflection:

- A transformation that produces a mirror image of an object.
- For 2D transformation, reflection is generated relative to an **axis of reflection** by rotating the object **180°**.



Reflection of an object about the y axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

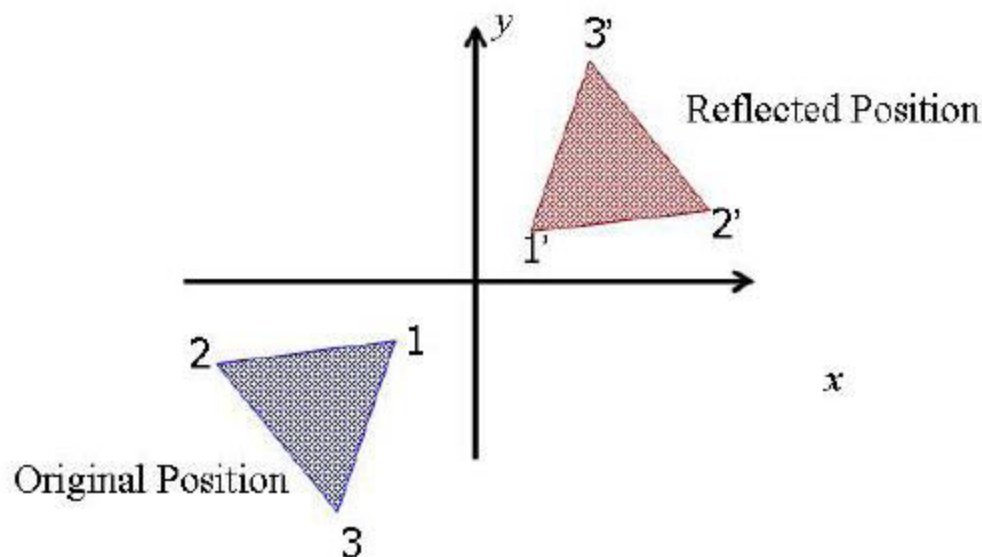
Transformation Matrix



Other two-dimensional transformation -3

■ Reflection:

- A transformation that produces a mirror image of an object.
- For 2D transformation, reflection is generated relative to an **axis of reflection** by rotating the object **180°**.



$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection of an object about coordinate origin

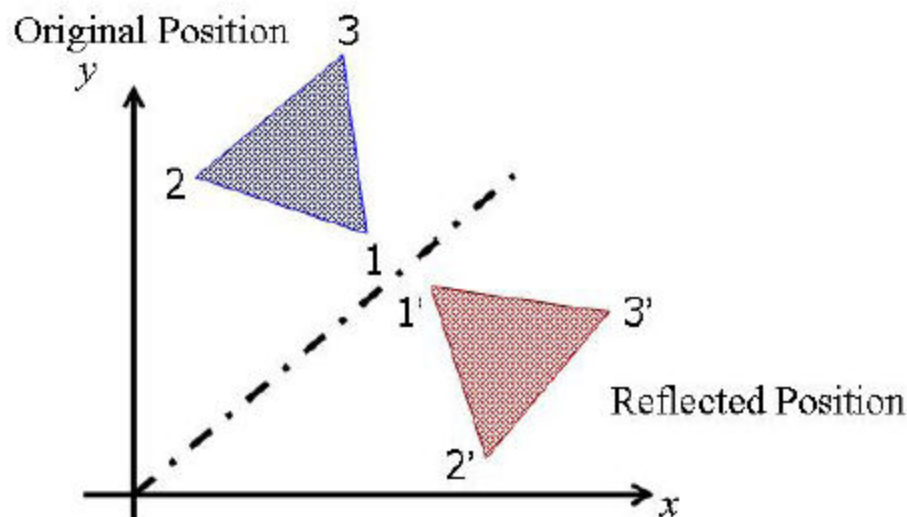
Transformation Matrix



Other two-dimensional transformation -4

■ Reflection:

- A transformation that produces a mirror image of an object.
- For 2D transformation, reflection is generated relative to an **axis of reflection** by rotating the object **180°**.



$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transformation Matrix

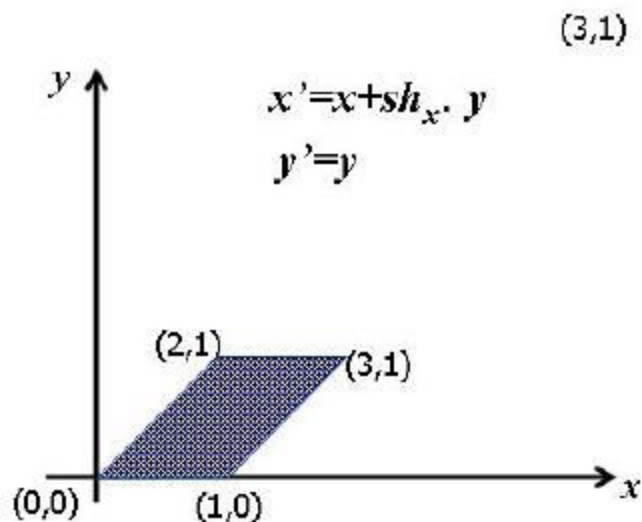
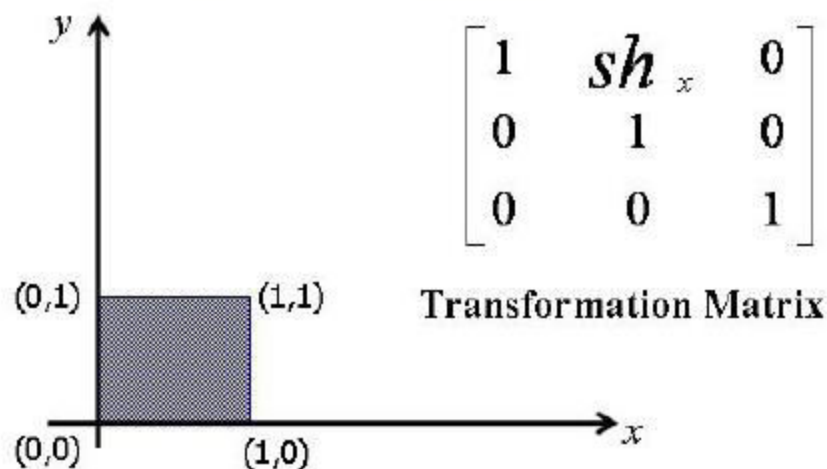
Reflection of an object about reflection axis $y=x$



Other two-dimensional transformation -5

■ Shear:

- A transformation that distorts the shape of an object.
- *Two common shearing transformations* are those that shift coordinate x values and those that shift y values:
 - An *x -direction* shear relative to *x axis* is produced with the transformation matrix.



A unit square (a) is converted to a parallelogram (b) using the *x -direction* shear matrix with $sh_x=2$



Map of the course

- ✓ Basic Transformations of the Plane
- ✓ Homogeneous Coordinates
- ✓ Composition of 2D Transformations
- ✍ **Basic Transformations of the Space**
 - **Composition of 3D Transformations**
 - **Hierarchical Modeling**



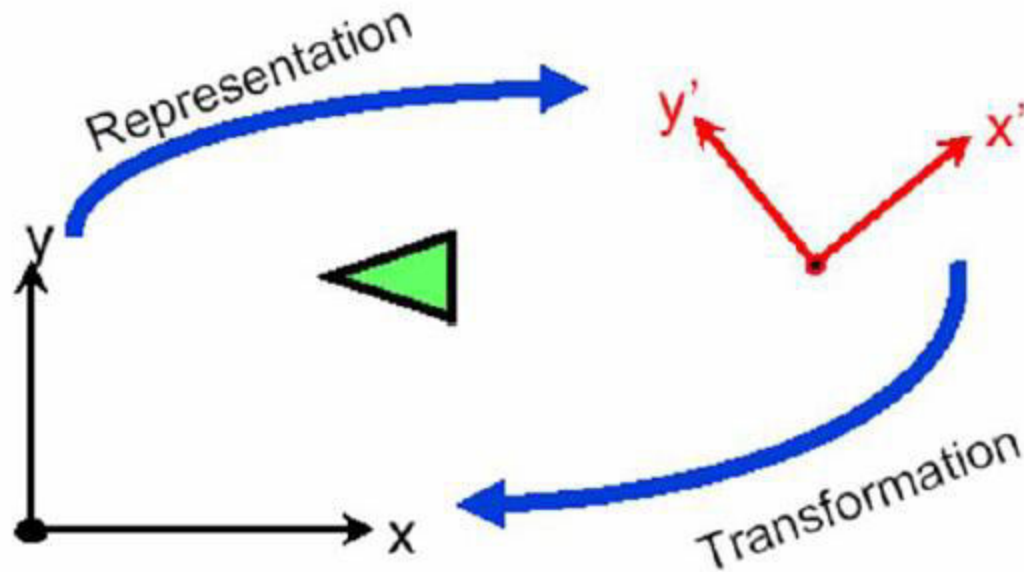
Change of Coordinates

- It is often required to transform the description of an object from one coordinate system to another.
- The viewing routines transform object descriptions from world coordinates to device coordinate.
- For modeling and design applications, individual objects are typically defined in their own local Cartesian references.
- These local-coordinate descriptions must then be transformed into positions and orientations within the overall scene coordinate system.
- Example: Office-layouts; chairs and tables can be placed into a floor plan, with multiple copies of the chairs and other items in different positions.



Change of Coordinates

- **Rule:** Transform one coordinate frame towards the other in the opposite direction of the representation change



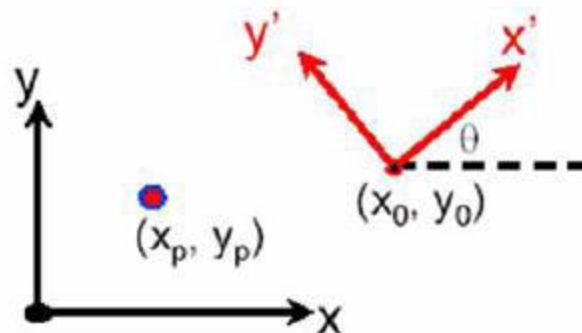
Change of Coordinates

- **Change of coordinates:** Represent $P = (x_p, y_p, 1)$ in the (x', y') coordinate system

$$P' = MP$$

Where:

$$M = \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{pmatrix}$$





Change of Coordinates

- In the upper-left 2*2 sub matrix, we consider each of the two rows as **vectors**.

$$M = \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Each a unit vector.
- Each is perpendicular to the other.
- The first and second vectors will be rotated to lie on the positive x and y axes respectively.
- A matrix having these properties is called **special orthogonal**.

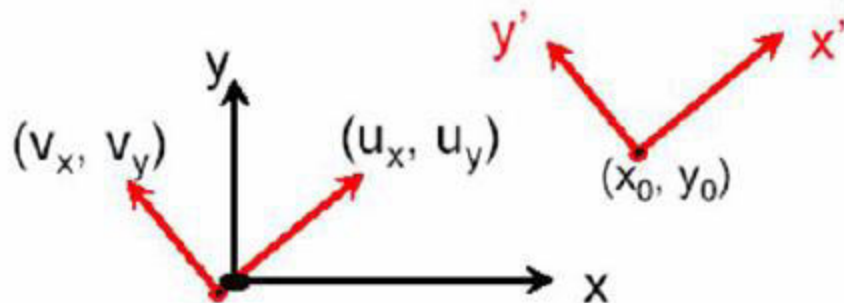
Change of Coordinates

- Change of coordinates:**

Alternative method: assume $x' = (u_x, u_y)$ and $y' = (v_x, v_y)$ in the (x, y) coordinate system

where $P' = MP$

$$M = \begin{pmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{pmatrix}$$





Efficiency

- The most general composition of **R,S,T** operations produce a matrix of the form:

$$M = \begin{bmatrix} \mathbf{r}_{11} & \mathbf{r}_{12} & \mathbf{t}_x \\ \mathbf{r}_{21} & \mathbf{r}_{22} & \mathbf{t}_y \\ 0 & 0 & 1 \end{bmatrix}$$

- The upper 2*2 sub matrix is a composite rotation and scale matrix
- Whereas t_x and t_y are composite translations
- The general matrix takes **9** multiplies and **6** adds,
- Whereas M reduce the process to **4** multiplies and **4** adds



Efficiency

- The rotation matrix is:

$$\begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \cos \theta + y \sin \theta \\ -x \sin \theta + y \cos \theta \end{bmatrix}$$

- When rotating of **small** angles θ , we can use the fact that $\cos(\theta) = 1$ and simplify:

$$\begin{bmatrix} 1 & \sin \theta \\ -\sin \theta & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + y \sin \theta \\ -x \sin \theta + y \end{bmatrix}$$

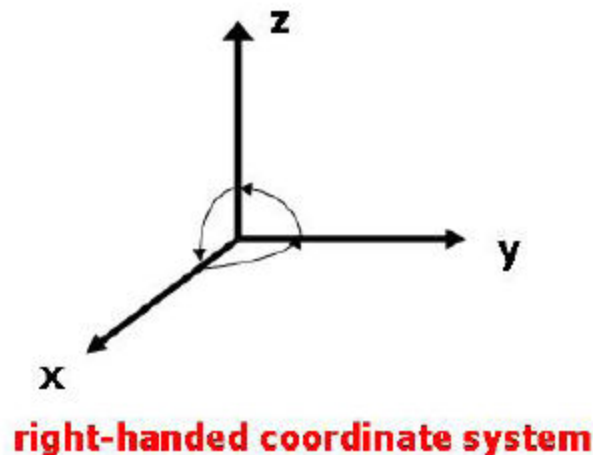
- A better approximation is to use x' instead of x in the second equation:
 - $x' = x - \sin \theta$
 - $y' = x \sin \theta + y(1 - \sin^2 \theta)$



Basic Transformations of the Space

■ **Convention:**

- The 3D coordinate system used is **right-handed**:
- **Positive rotation def.:**
 - When looking from a **positive** axis toward the origin,
 - A 90° **counterclockwise** rotation
 - Will transform one positive axis into the other.



If axis rotation is	Direction of positive rotation
X	Y to z
Y	Z to x
z	X to y

Left-handed coordinate system



Homogeneous coordinates in 3D

- In order to model all transformations as matrices:

- Introduce a fourth coordinate, w $\longrightarrow$

- Two vectors are equal if:

- $x/w = x'/w'$, $y/w = y'/w'$ and $z/w = z'/w'$

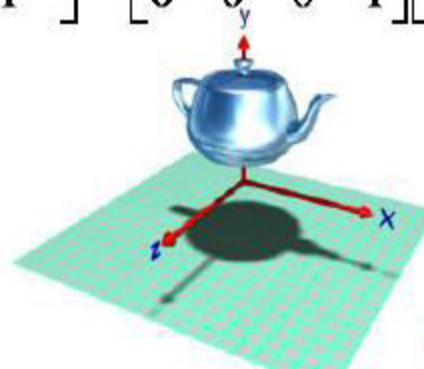
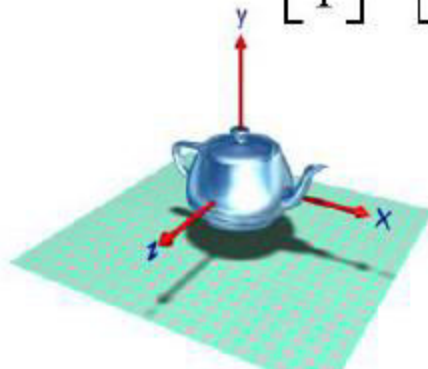
$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

- All transformations are 4x4 matrices

Basic Transformations of the Space

- **3D translation:** A simple extension from that in 2D:
 - Translation *only applies to points*, **never translate vectors**
 - Remember: points have homogeneous co-ordinate $w = 1$

$$\begin{aligned}x' &= x + a \\y' &= y + b \\z' &= z + c\end{aligned} \quad \Rightarrow \quad \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} x + a \\ y + b \\ z + c \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



translate along y



Basic Transformations of the Space

■ Scaling in 3D:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \\ s_z z \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \Rightarrow \mathbf{v}' = \mathbf{S}\mathbf{v}$$

We would also like to scale points thus we need a *homogeneous transformation* for consistency:

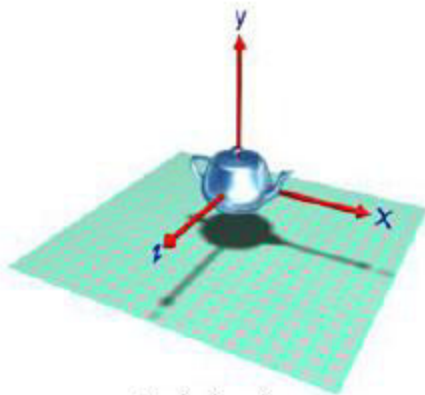
$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \\ s_z z \\ w \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \quad \mathbf{S}^{-1} = \begin{bmatrix} 1/s_x & 0 & 0 & 0 \\ 0 & 1/s_y & 0 & 0 \\ 0 & 0 & 1/s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Point Transformation

Point Transformation

Basic Transformations of the Space

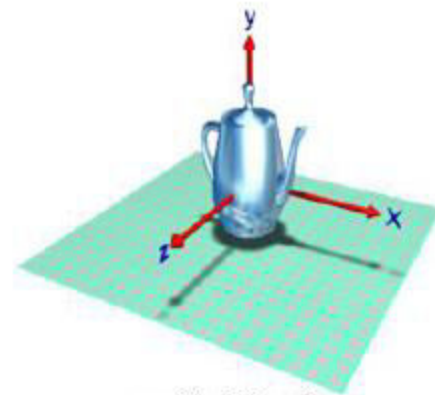
■ Scaling in 3D:



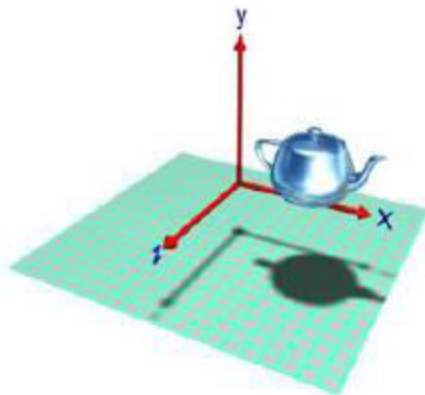
Original



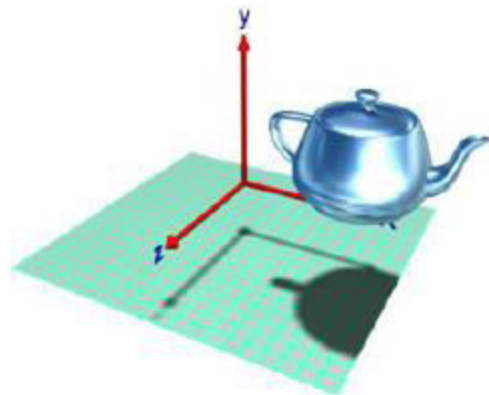
scale all axes



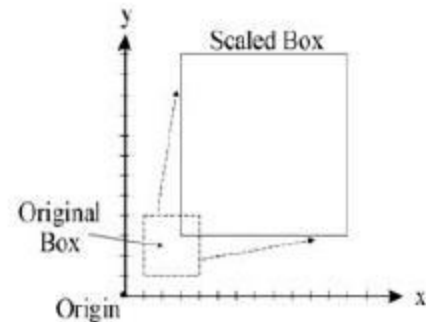
scale Y axis



offset from origin



distance from origin also scales



Basic Transformations of the Space

- **Rotations in 3D:** 3D homogeneous rotations:

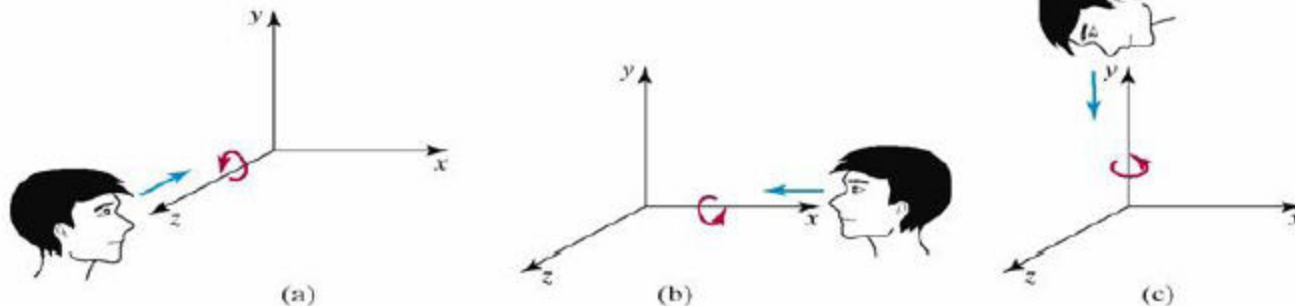


Figure 5-36

Positive rotations about a coordinate axis are counterclockwise, when looking along the positive half of the axis toward the origin.

3D rotation:

One rotation: one axis and one angle

Matrix depends on both axis and angle

Basic Transformations of the Space

- **Rotations in 3D:** 3D homogeneous rotations:

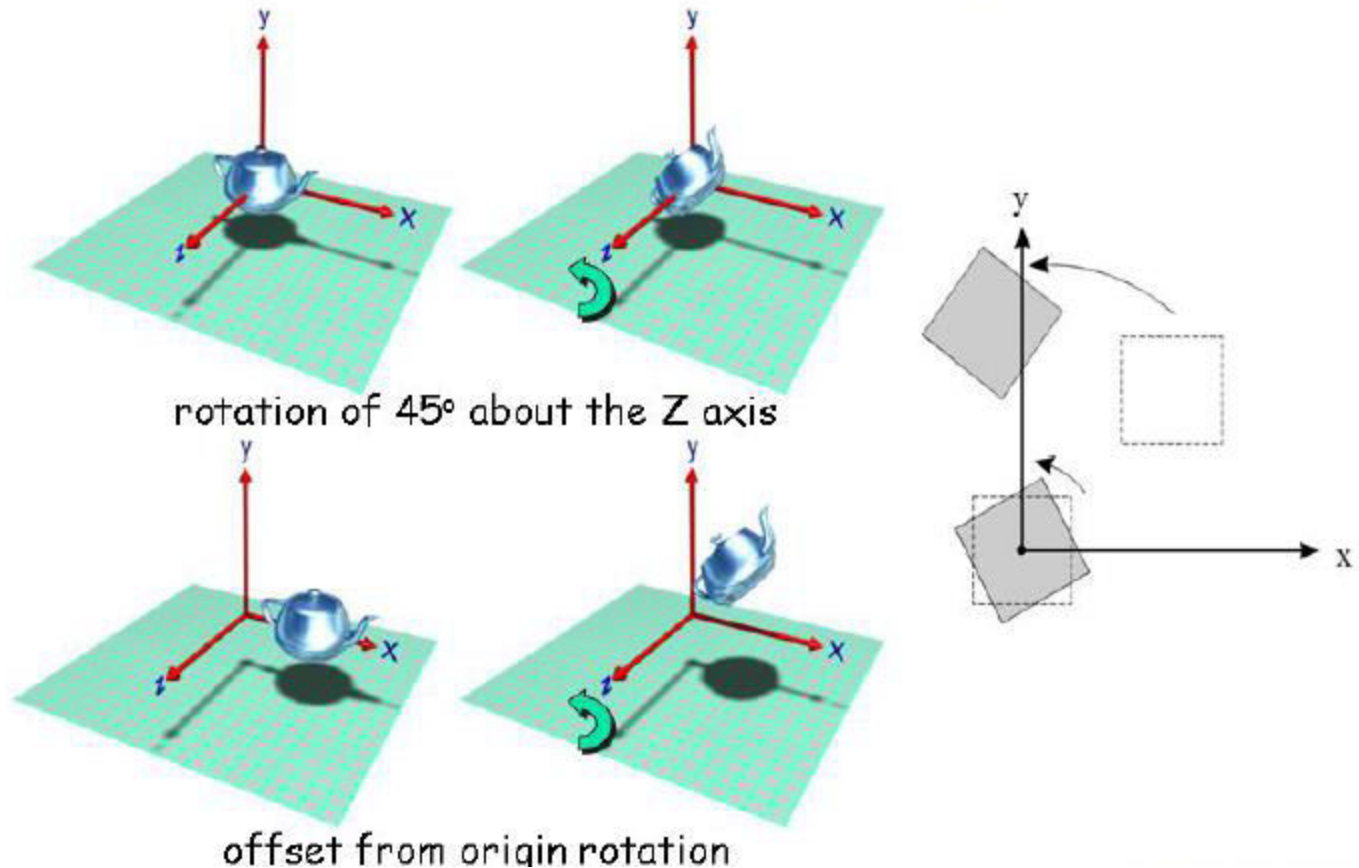
$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}_y = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}_z = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Note: $\begin{matrix} \cos(-\theta) = \cos \theta \\ \sin(-\theta) = -\sin \theta \end{matrix} \Rightarrow \mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta) = \mathbf{R}^T(\theta)$
- If $\mathbf{M}^{-1} = \mathbf{M}^T$ then $\mathbf{M}$ is *orthonormal*. All orthonormal matrices are rotations about the origin.



Basic Transformations of the Space

- Rotations are *anti-clockwise* about the *origin*:

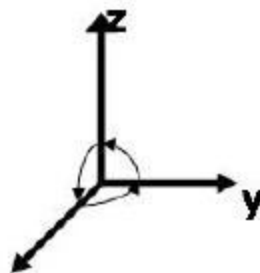


Basic Transformations of the Space

■ Rotations in 3D:

■ Rotation around x-axis:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R_x\left(\frac{\pi}{2}\right) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sanity check: a rotation of $\pi/2$ should change y in z , and z in $-y$

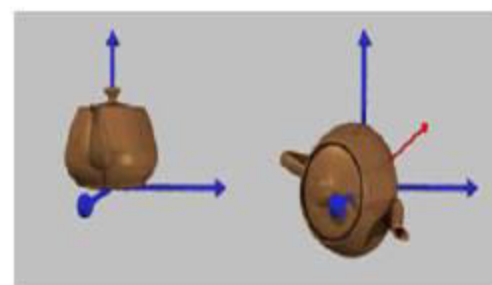
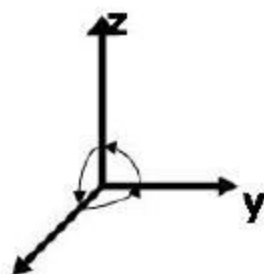


Basic Transformations of the Space

■ Rotations in 3D:

■ Rotation around y-axis:

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R_y\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sanity check: a rotation of $\pi/2$ should change z in x , and x in $-z$

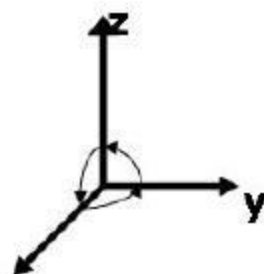


Basic Transformations of the Space

■ Rotations in 3D:

■ Rotation around z -axis:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R_z\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

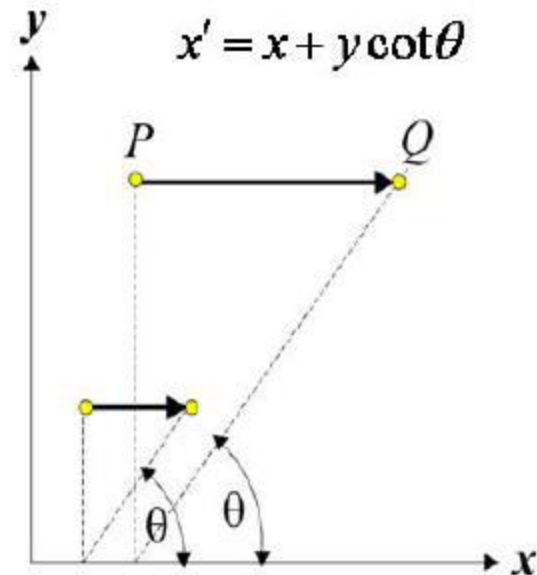
Sanity check: a rotation of $\pi/2$ should change x in y , and y in $-x$



Basic Transformations of the Space

■ Shear

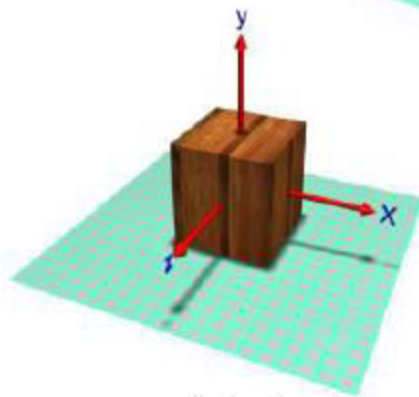
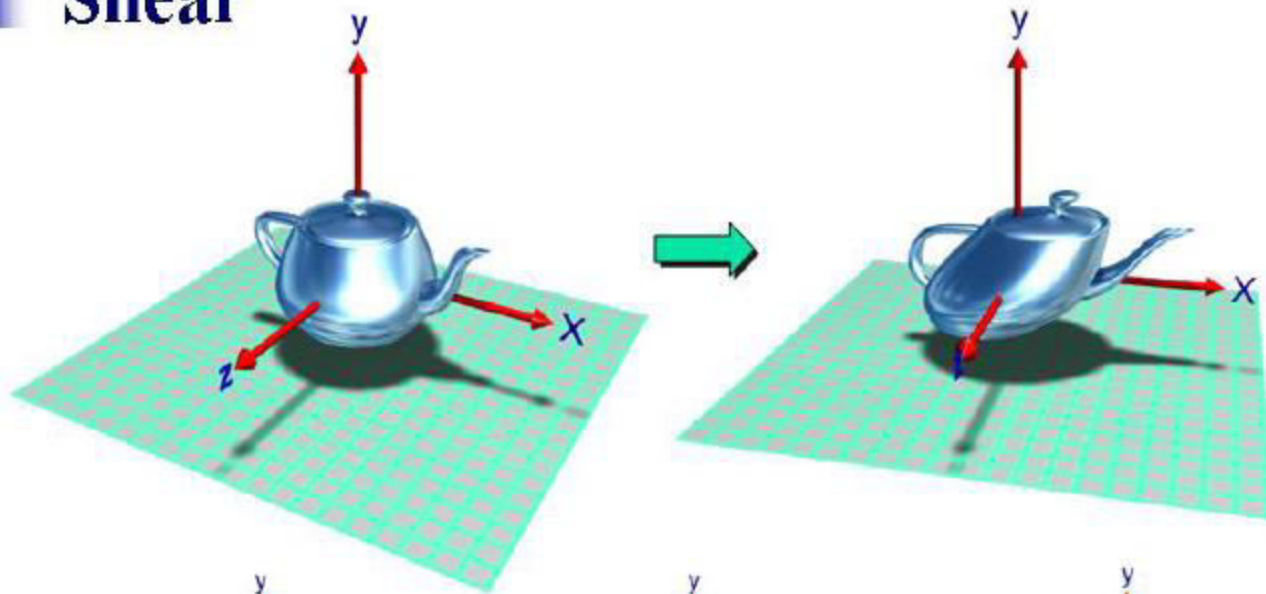
- We shear *along* an axis according to another axis
 - Shearing along X axis preserves y and z values
 - Shearing along Y axis preserves x and z values
 - Shearing along Z axis preserves x and y values
- Point are stretched along the shear axis in proportion to the distance of the point along another axis
- **Example:** shearing along X according to Y



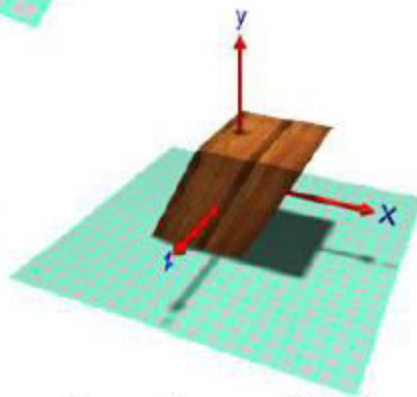
$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & \cot \theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$



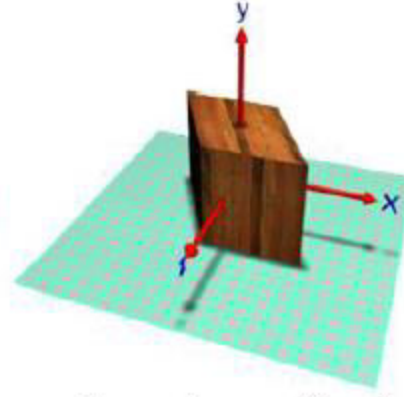
Shear



original



shear along x (by y)



shear along x (by z)



Map of the course

- ✓ Basic Transformations of the Plane
- ✓ Homogeneous Coordinates
- ✓ Composition of 2D Transformations
- ✓ Basic Transformations of the Space
- ✗ Composition of 3D Transformations
- Hierarchical Modeling



Composition of 3D Transformations

- A rotation matrix for any axis **that does not coincide** with a coordinate axis can be set up as a composite transformation involving combinations of:
 - Translations and
 - The coordinate axis rotations.
- 1. We first move the designated rotation axis one of the coordinate axis.
- 2. Then, we apply the appropriate rotation matrix for that coordinate axis.
- 3. The last step is to return the rotation axis to its original axis.



Composition of 3D Transformations

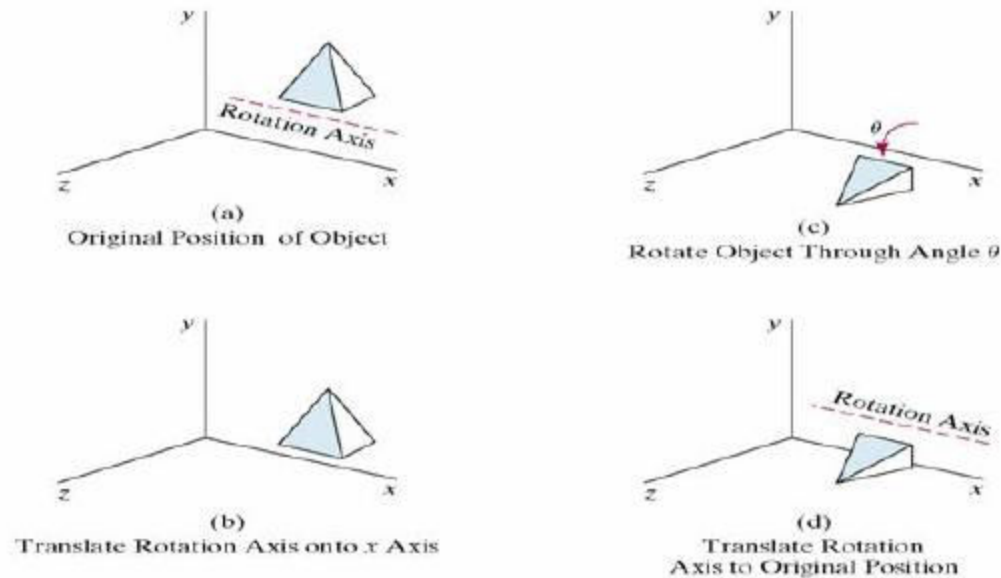


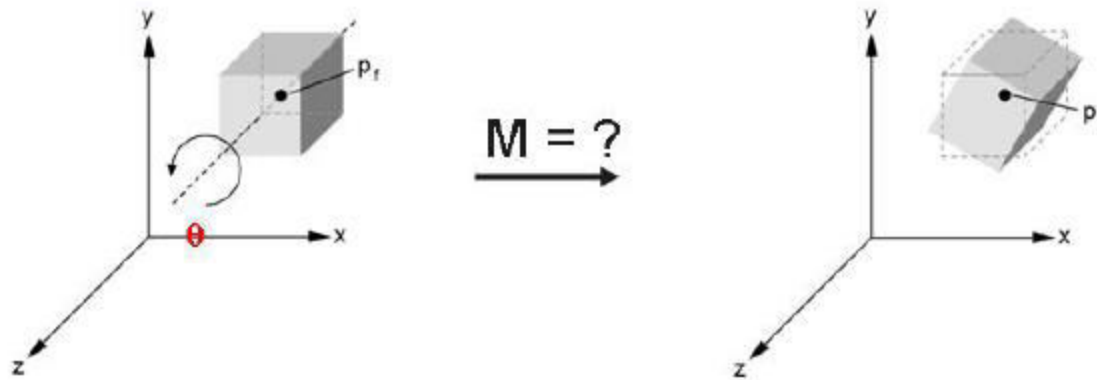
Figure 5-41

Sequence of transformations for rotating an object about an axis that is parallel to the x axis.

$$P = T \cdot Rx(\theta) \cdot T^{-1} \cdot P$$

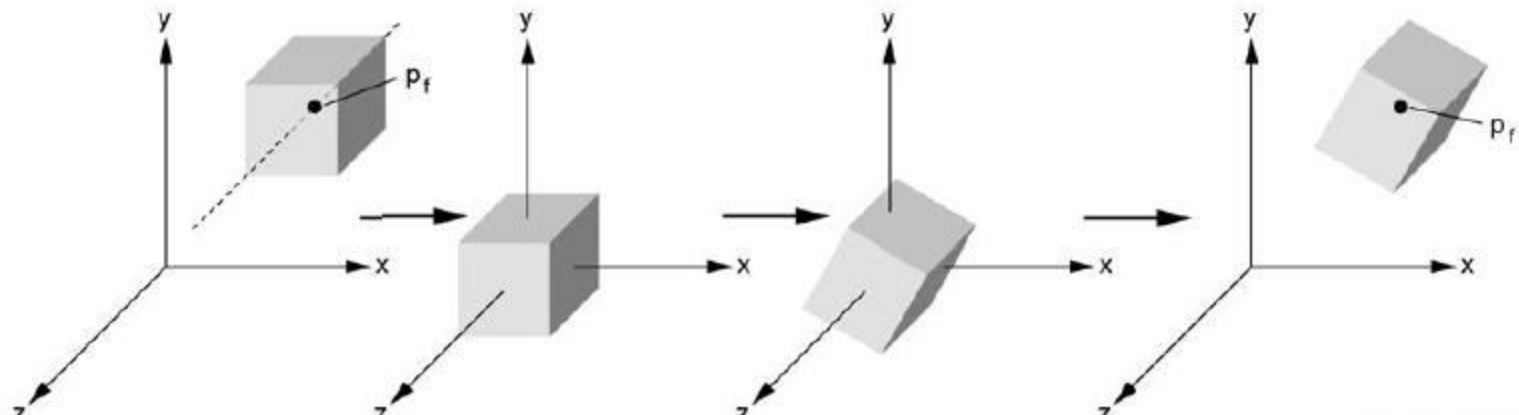


Composition of 3D Transformations



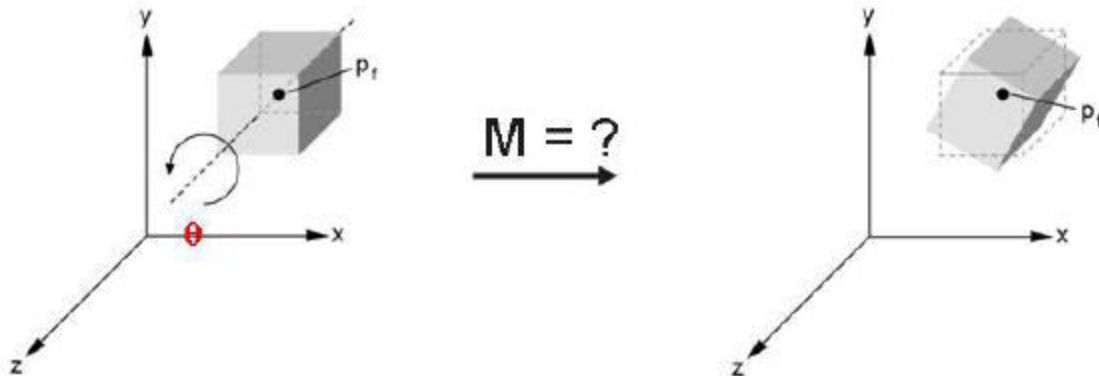
Rotation about a Fixed Point p_f around the z -axis by θ

(center of cube at p_f , sides aligned with axes)





Composition of 3D Transformations

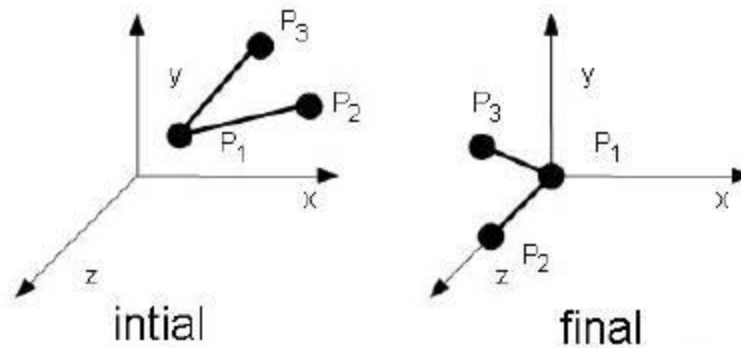


Rotation about a Fixed Point p_f around the z -axis by θ
(center of cube at p_f , sides aligned with axes)

$$\mathbf{M} = \mathbf{T}(p) \mathbf{R}_z(\theta) \mathbf{T}(-p)$$



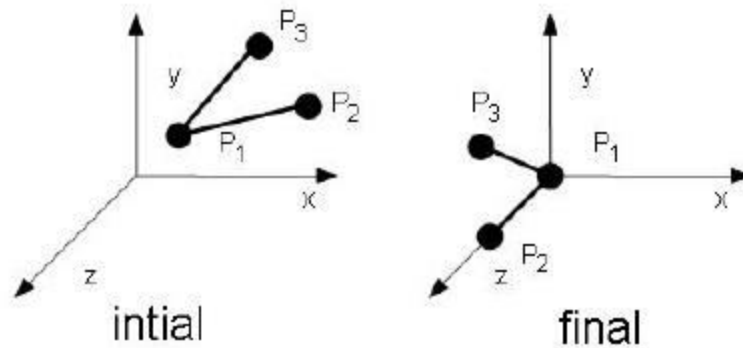
Composition of 3D Transformations



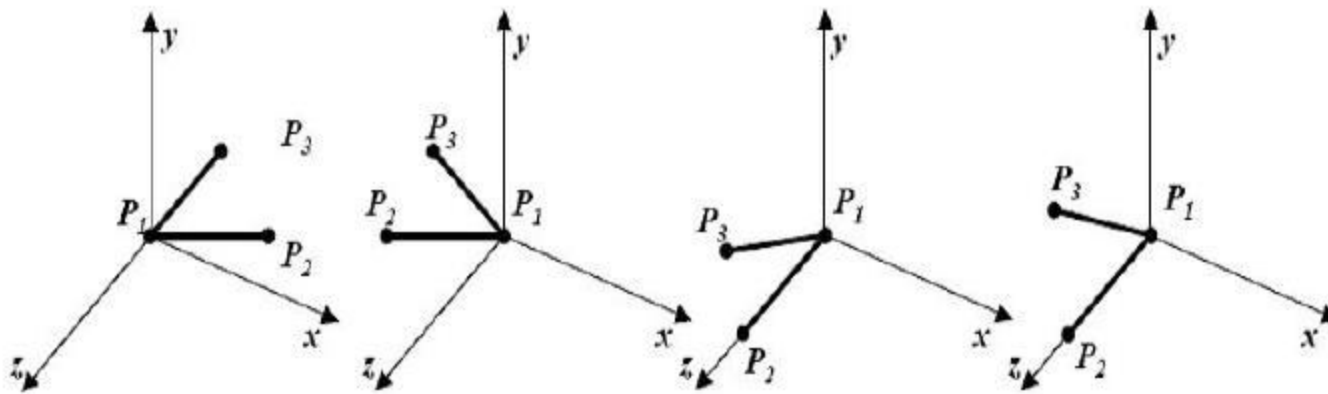
Transform the directed line segments P_1P_2 , P_2P_3 and P_3P_1



Composition of 3D Transformations

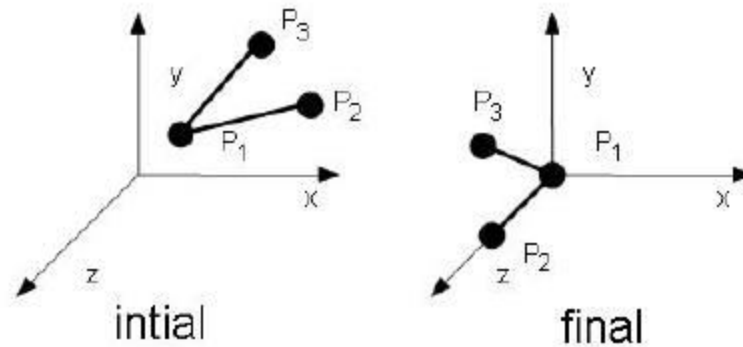


Transform the directed line segments P_1P_2 , P_2P_3 and P_3P_1





Composition of 3D Transformations



Transform the directed line segments P_1P_2 , P_2P_3 and P_3P_1

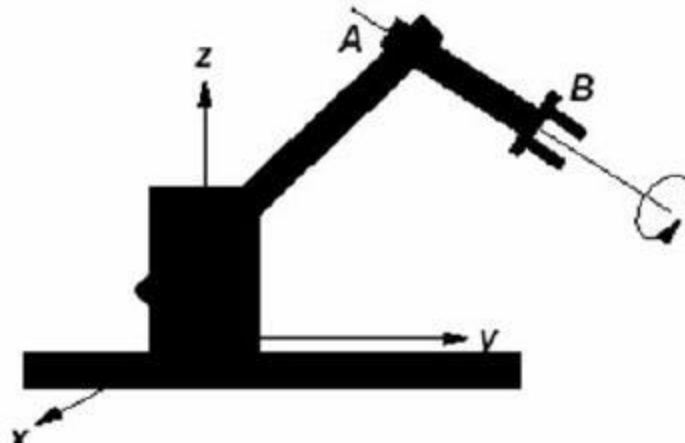
1. Translate P_1 to the origin
2. Rotate about the **y-axis** such that P_1P_2 lies in the y-z plane
3. Rotate about the **x-axis** such that P_1P_2 lies on the z-axis
4. Rotate about the **z-axis** such that P_1P_3 lies in the y-z plane



Composition of 3D Transformations

■ Rotation about Arbitrary Axes:

- Rotations about an arbitrary axis in space are frequently found in engineering applications.
- A rotation matrix for **any axis that does not coincide with a coordinate axis** can be set up as a composite transformation involving combination of translations and the coordinate-axes rotations.





Composition of 3D Transformations

■ **Rotation about Arbitrary Axes:**

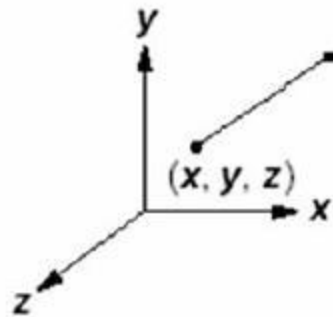
- **Translate** the object so that the rotation axis passes through the coordinate origin
- **Rotate** the object so that the axis rotation **coincides** with **one** of the coordinate axes
- Perform the **specified rotation** about that coordinate axis
- Apply **inverse rotation** axis back to its original orientation
- Apply the **inverse translation** to bring the rotation axis back to its original position



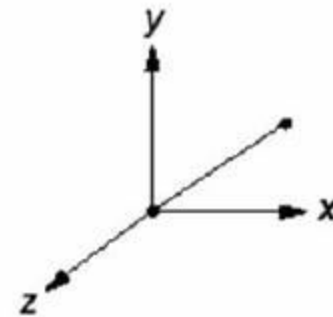
Composition of 3D Transformations

■ Rotation about Arbitrary Axes:

- **Step1:** Translate the arbitrary axis so that one of its end points coincides with the origin.



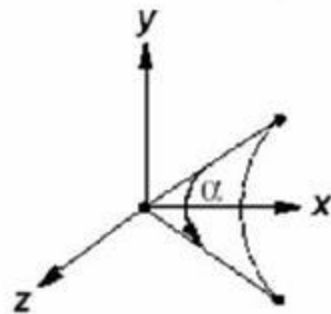
$$T_{tr} = \begin{bmatrix} 1 & 0 & 0 & -x \\ 0 & 1 & 0 & -y \\ 0 & 0 & 1 & -z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Composition of 3D Transformations

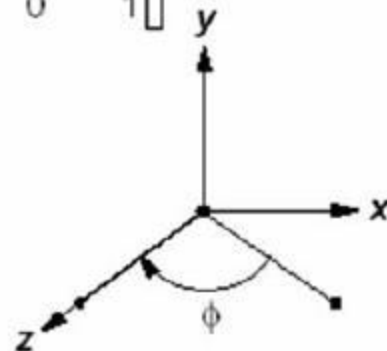
■ Rotation about Arbitrary Axes:

- **Step2:** Rotate about the x and y axes to align the arbitrary axis with the positive z -axis.



$$T_{rx}(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{ry}(\phi) = \begin{bmatrix} \cos \phi & 0 & -\sin \phi & 0 \\ 0 & 1 & 0 & 0 \\ \sin \phi & 0 & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





Composition of 3D Transformations

- **Rotation about Arbitrary Axes:**
 - **Step3:** Rotate about the z -axis by the desired angle θ

$$T_{rz}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Composition of 3D Transformations

■ Rotation about Arbitrary Axes:

- Step 4 and step 5: use the similar transformations as steps 2 and 1 respectively.

$$T_{\text{arb}} = T_{\text{tr}}(x, y, z) \cdot T_{\text{rx}}(-\alpha) \cdot T_{\text{ry}}(-\phi) \cdot T_{\text{rz}}(\theta) \cdot T_{\text{ry}}(\phi) \cdot T_{\text{rx}}(\alpha) \cdot T_{\text{tr}}(-x, -y, -z)$$



Map of the course

- ✓ Basic Transformations of the Plane
- ✓ Homogeneous Coordinates
- ✓ Composition of 2D Transformations
- ✓ Basic Transformations of the Space
- ✓ Composition of 3D Transformations
- ✗ **Hierarchical Modeling**



Introduction

- Explore multiple approaches to developing and working with models of geometric objects.
- Extend the use of transformations from to include hierarchical relationships among the objects.
- The techniques that we develop are appropriate for applications, such as **robotics** and figure **animation**, where the **dynamic behavior** of the objects is **characterized by relationships among the parts of the model**.



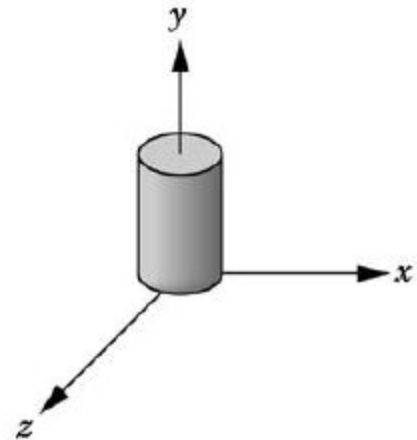
Symbols and Instances

- Our first concern is how to store a model that may include many sophisticated objects.
- There are two immediate issues:
 - **How do we define a complex object**
 - **How do we represent a collection of these objects.**
- We can take a non hierarchical approach to modeling regarding these objects as **symbols**, and modeling our world as a **collection of symbols**.



Symbols and Instances

- Symbols are usually represented at a convenient size and orientation.
 - For example: a cylinder

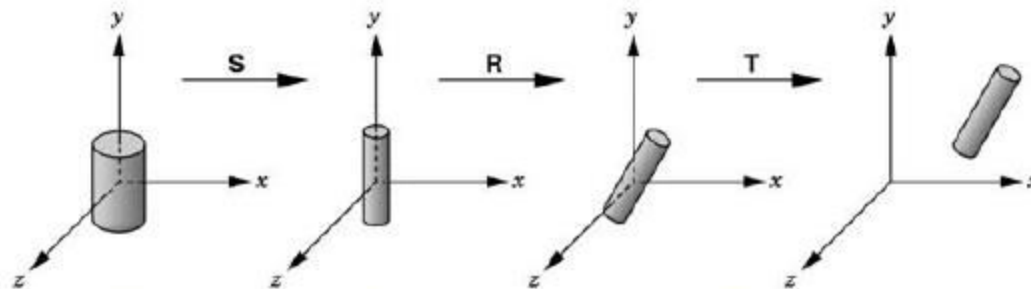


- In OpenGL we have to set up the *appropriate transformation from the frame of the symbol to the world coordinate frame* to apply it to the model-view matrix before we execute the code for the symbol.



Symbols and Instances

- For example: the instance transformation
 - $M = TRS$
 - is a concatenation of a translation, a rotation, and a scale



- And the OpenGL program often contains this code:
 - `glMatrixMode(GL_MODELVIEW);`
 - `glLoadIdentity();`
 - `glTranslatef(...);`
 - `glRotatef(...);`
 - `glScalef(...);`
 - `glutSolidCylinder(...);`



Symbols and Instances

- We can also think of such a model in the form of a table

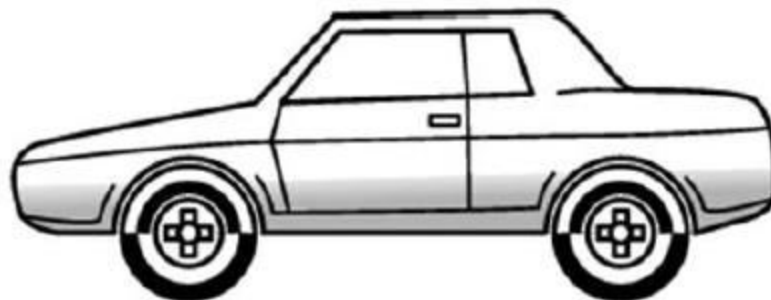
Symbol	Scale	Rotate	Translate
1	s_x, s_y, s_z	u_x, u_y, u_z	d_x, d_y, d_z
2			
3			
1			
1			
.			
.			

- The table shows **no relationship among the objects**.
- However, it does contain everything we need to draw the objects
- We could do a lot of things with this data structure, but its flatness limits us.



Symbols and Instances

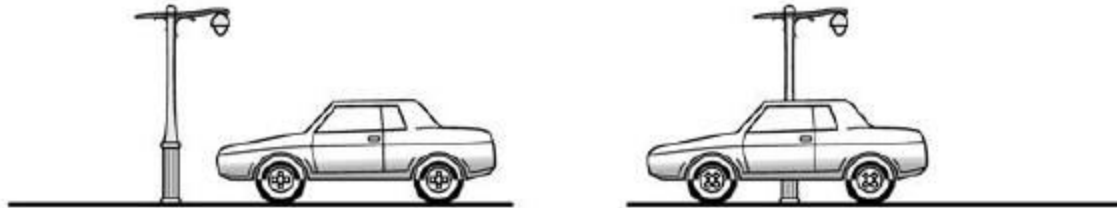
- Suppose we wish to build an automobile that we can animate
- We can build it from the chassis, and 4 wheels





Symbols and Instances

- Two frames of our animation could look like this:



- We could calculate how far the wheel moved, and have code that looks like
 - `calculate(speed, distance);`
 - `draw_right_front_wheel(speed, dist);`
 - `draw_left_front_wheel(speed, dist);`
 - `draw_right_rear_wheel(speed, dist);`
 - `draw_left_rear_wheel(speed, dist);`
 - `draw_chassis(speed, dist);`

Fails to show relationships well



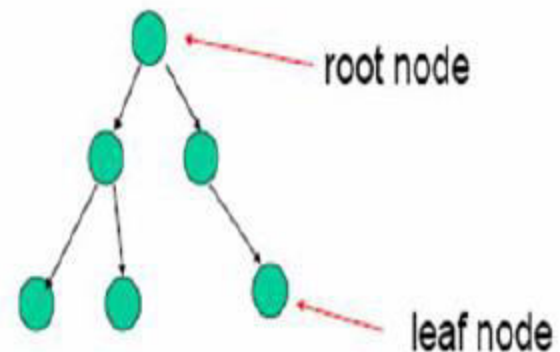
Hierarchical Models

- But this is the kind of code we do NOT want....
 - It is linear, and it shows none of the relationships between the 5 objects.
- There are two types of relationships we wish to convey
 - **First**, we cannot separate the movement of the car from the movement of the wheels
 - If the car moves forward, the wheels must turn.
 - **Second**, we would like to note that all the wheels are identical and just located and oriented differently.
 - We typically do this with a graph.
 - Def: node, edge, root, leaf, ...



Hierarchical Models

- Graph: Set of *nodes* and *edges (links)*
 - Edge connects a pair of nodes
 - Directed or undirected
- Tree:
 - Graph in which each node (except the root) has exactly one parent node
 - May have multiple children
 - Leaf or terminal node: no children



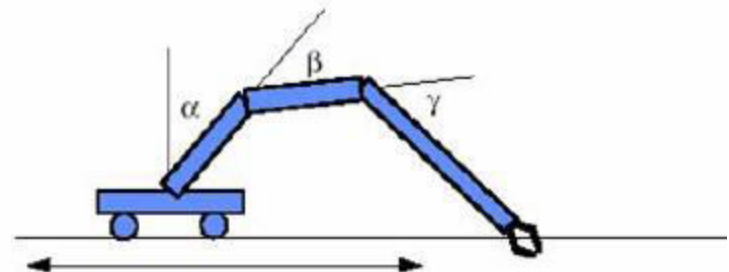


Hierarchical Models

- Integrated In Graphic Systems:
 - PHIGS
 - Stores information in a special-purpose database called the *central structure storage*.
 - Open GL, Renderman
 - Hierarchy procedures calling
 - Open Inventor, WTK, Java3D, VRML
 - Hierarchy objects calling

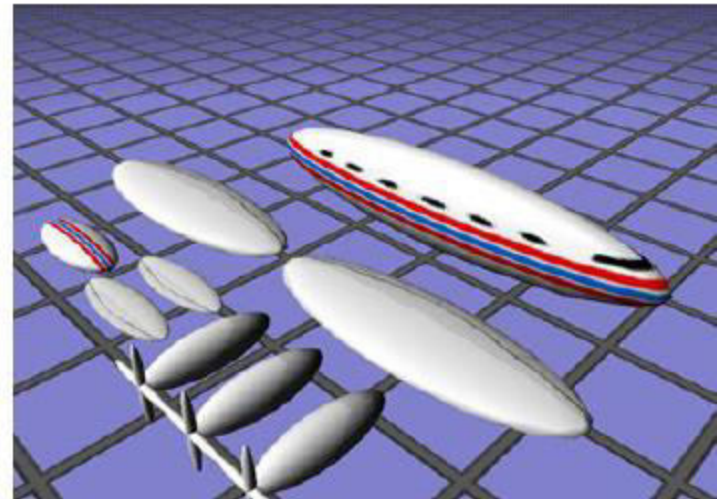
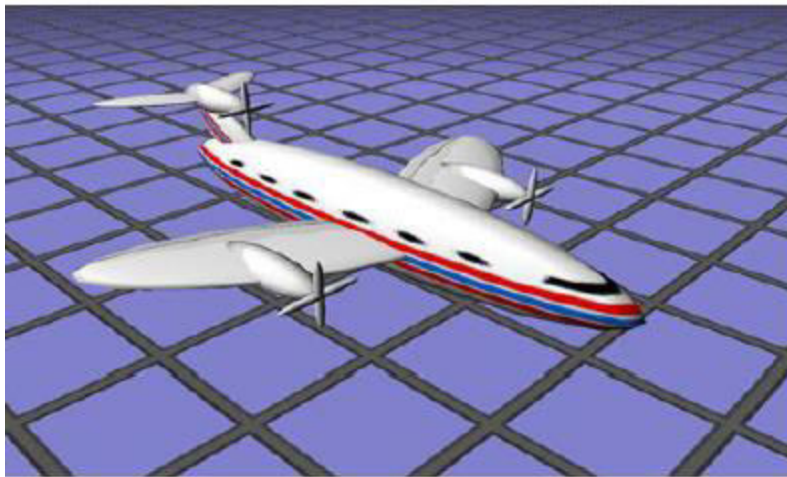
Hierarchical Modeling

- A hierarchy is created for a variety of purposes:
 - To construct complex objects.
 - To increase storage economy:
 - Store only references to objects that are used repeatedly.
 - To allow easy update propagation
 - A change in the definition of one object is propagated to all lower level objects that use that object.
 - Ensure a **consistent** behavior
 - Object stays **connected**



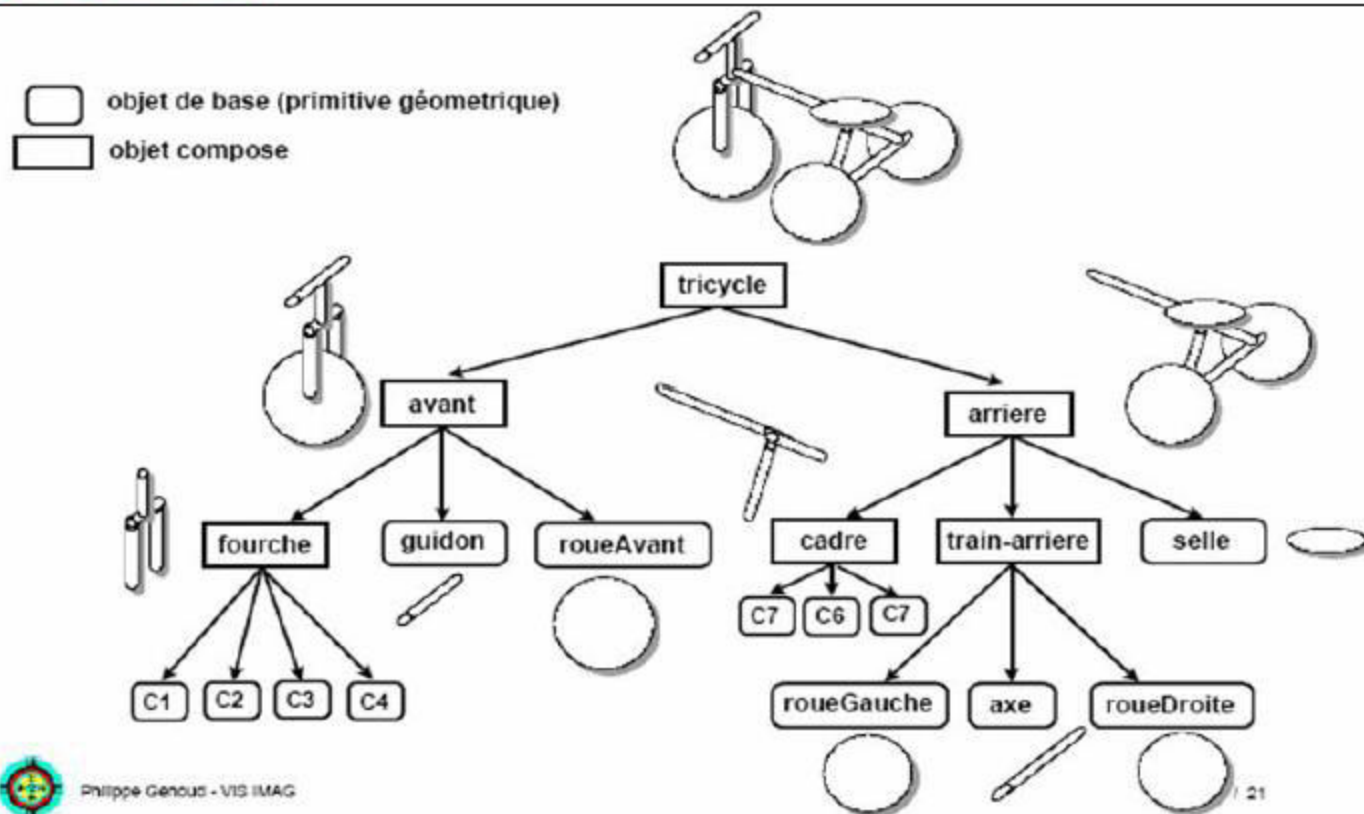
Hierarchical Modeling

- Geometric models often have a hierarchical structure induced by a **bottom-up** construction process.



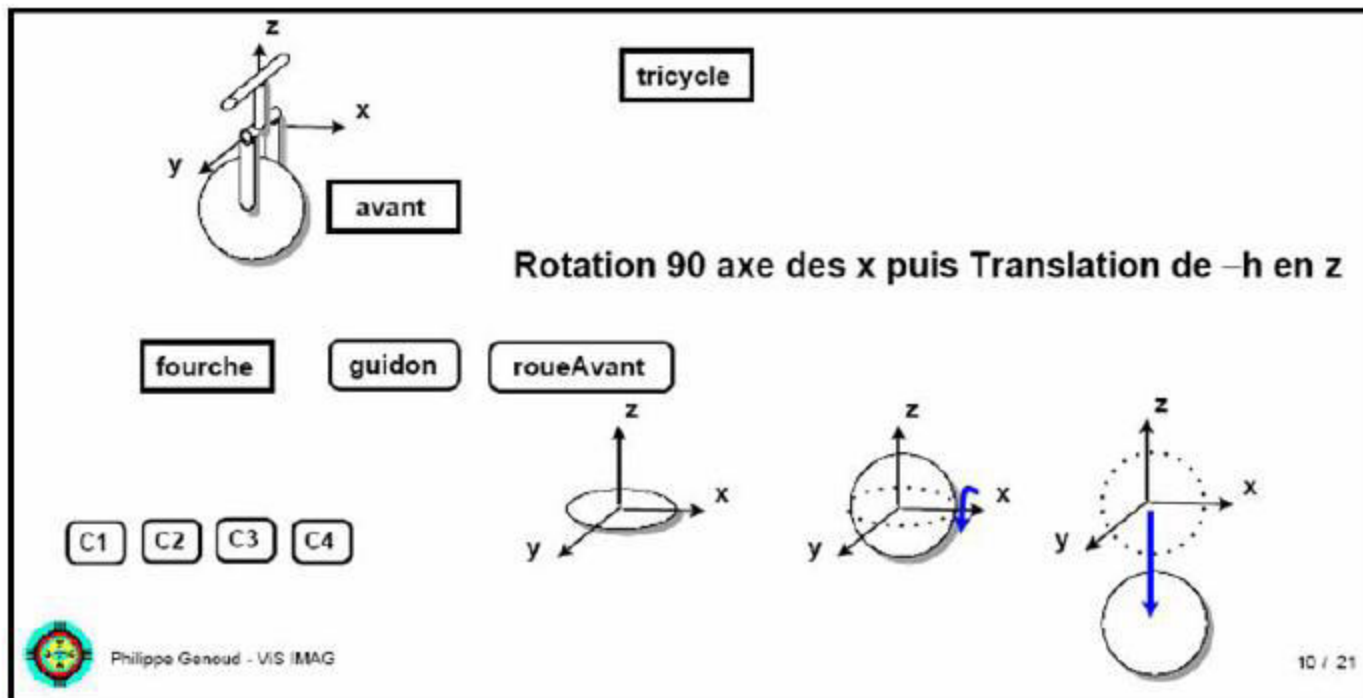
Hierarchical Modeling

- In the more common case of objects included multiple times, the hierarchy is symbolized by a **directed acyclic graph (DAG)**.



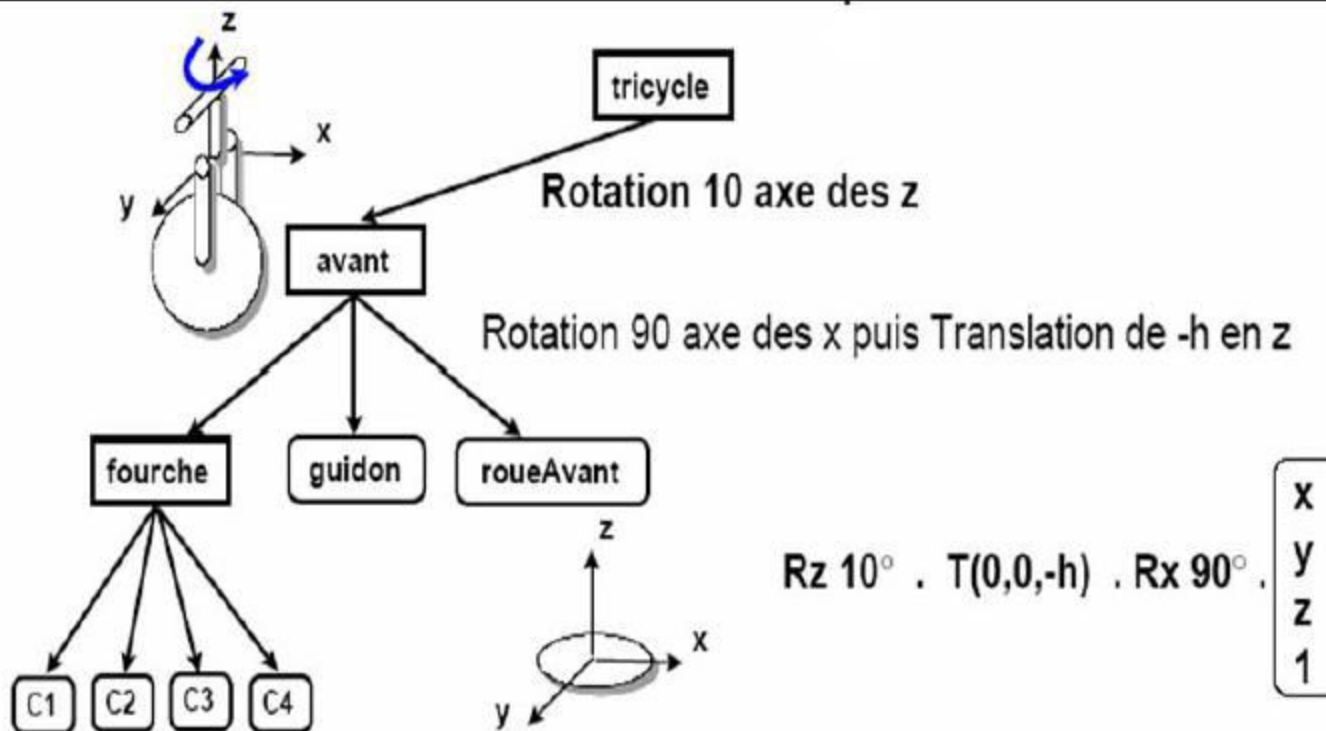
Hierarchical Modeling

- Primitives objects with **basic defaults** (located at the origin with unit area of volume) are stored as **nodes** in the graph
- They are then **transformed** and placed in their proper place in the scene: **Translation, Rotating, Scaling.**



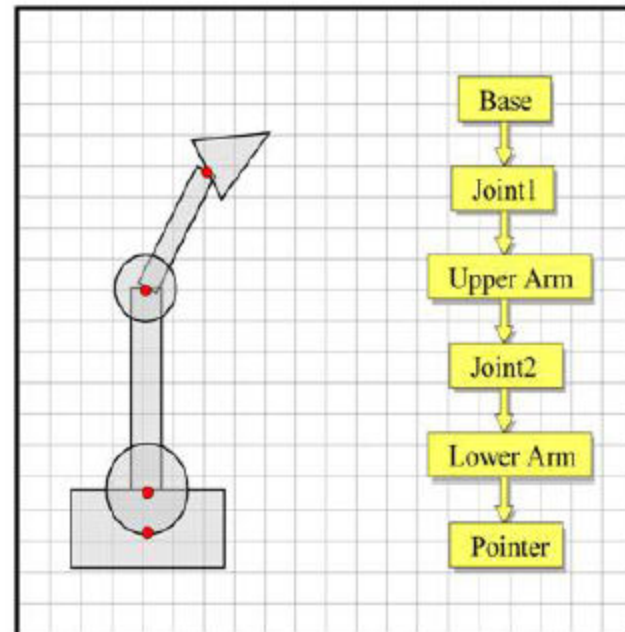
Hierarchical Modeling

- **Parent** object **call** a **child** object in a hierarchy, and **passing** it **geometric parameters** corresponding to its scale, orientation, and position in the parent's coordinate system.



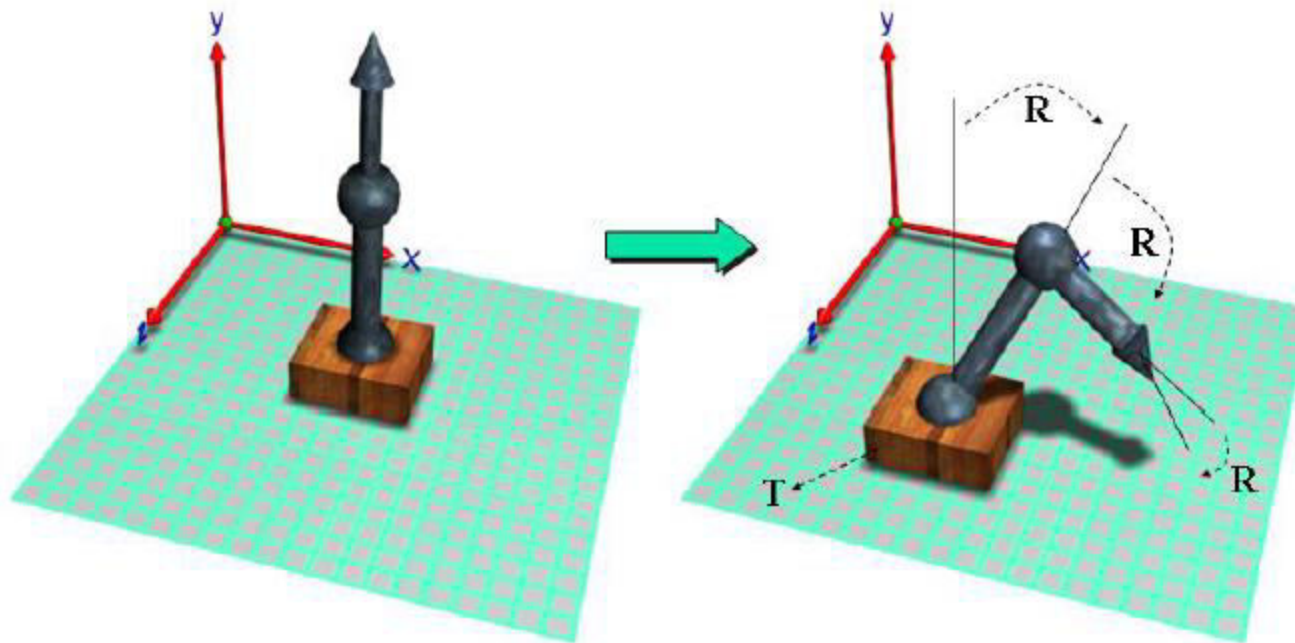
Hierarchical Transformations: A Robot Arm

- For geometries with an implicit *hierarchy* we wish to associate local frames with sub-objects in the assembly.
- *Parent-child frames* are related via a transformation.
- Transformation linkage is described by a *tree*:
- Each node has its own *local coordinate system*.

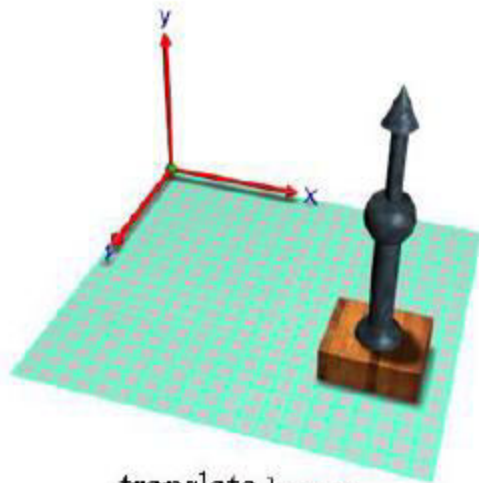




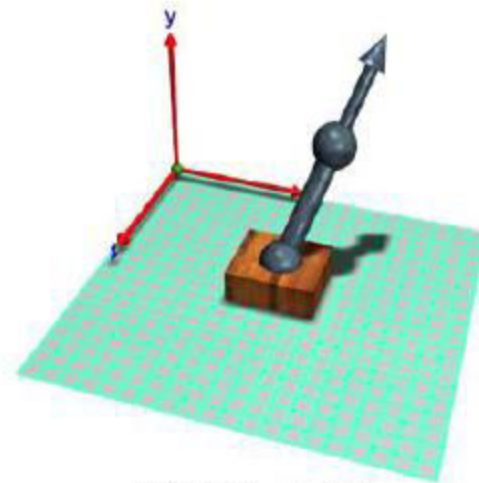
Hierarchical Transformations: A Robot Arm



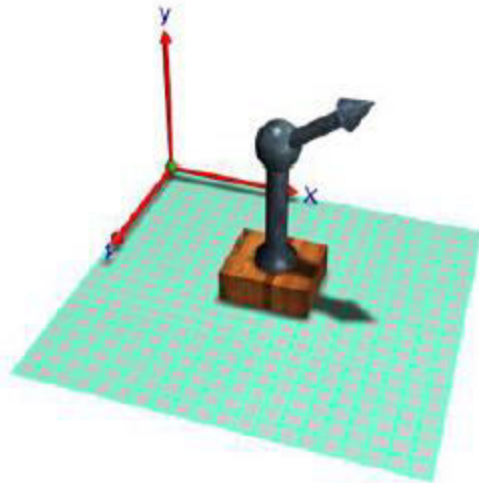
Hierarchical transformation allow independent control over sub-parts of an assembly



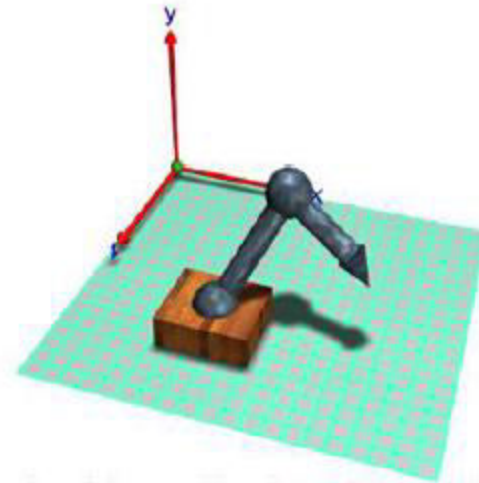
translate base



rotate joint1



rotate joint2

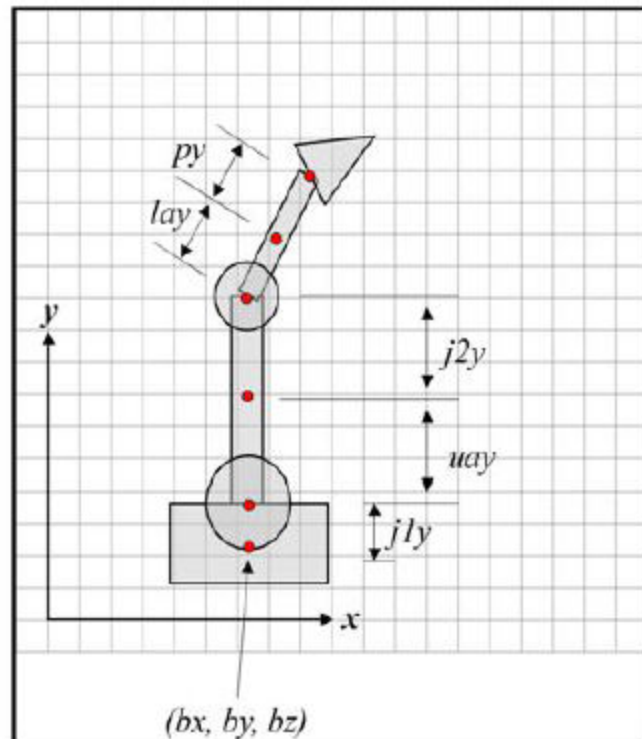


complex hierarchical transformation



OpenGL® Implementation

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
glTranslatef(bx, by, bz);  
    create_base();  
glTranslatef(0, j1y, 0);  
glRotatef(joint1_orientation);  
    create_joint1();  
glTranslatef(0, uay, 0);  
    create_upperarm();  
glTranslatef(0, j2y);  
glRotatef(joint2_orientation);  
    create_joint2();  
glTranslatef(0, lay, 0);  
    create_lowerarm();  
glTranslatef(0, py, 0);  
glRotatef(pointer_orientation);  
    create_pointer();
```



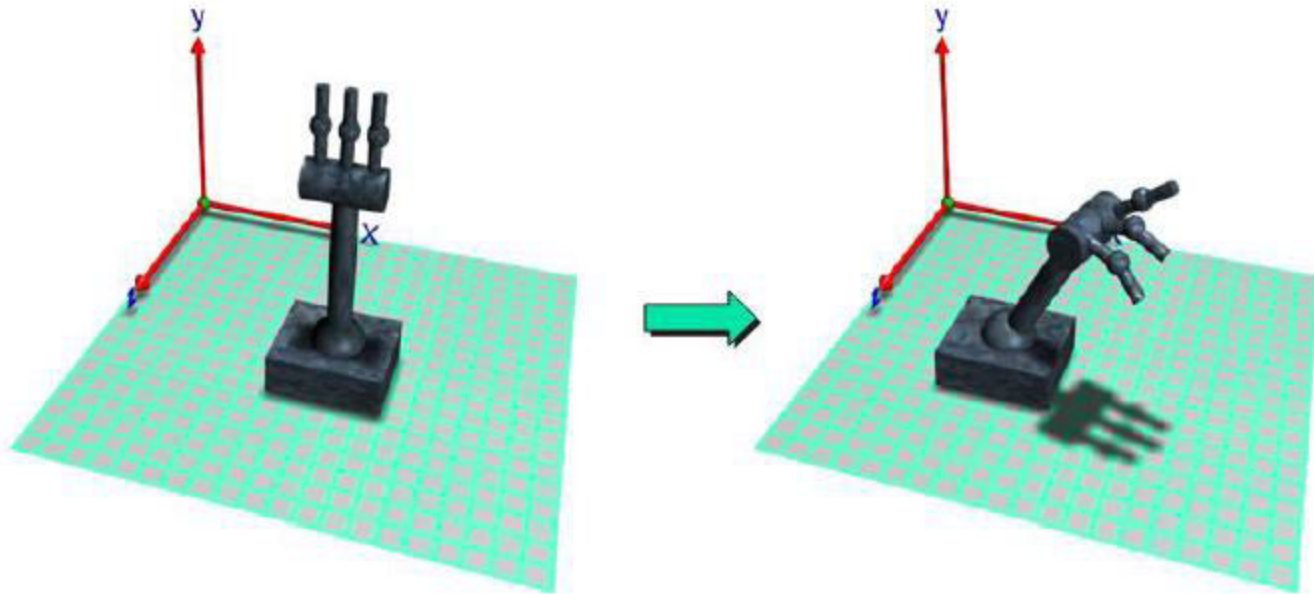


Hierarchical Transformations

- The previous example had simple *one-to-one* parent-child linkages.
- In general there may be many *child frames* derived from a single parent frame.
- we need some mechanism to remember the parent frame and return to it when creating new children.
- OpenGL provide a matrix stack for just this purpose:
 - `glPushMatrix()` saves the *CTM*
 - `glPopMatrix()` returns to the last saved *CTM*



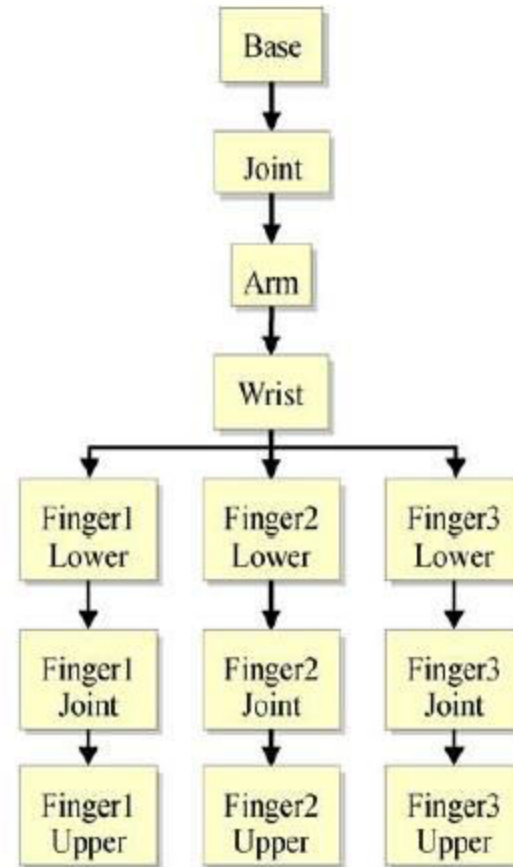
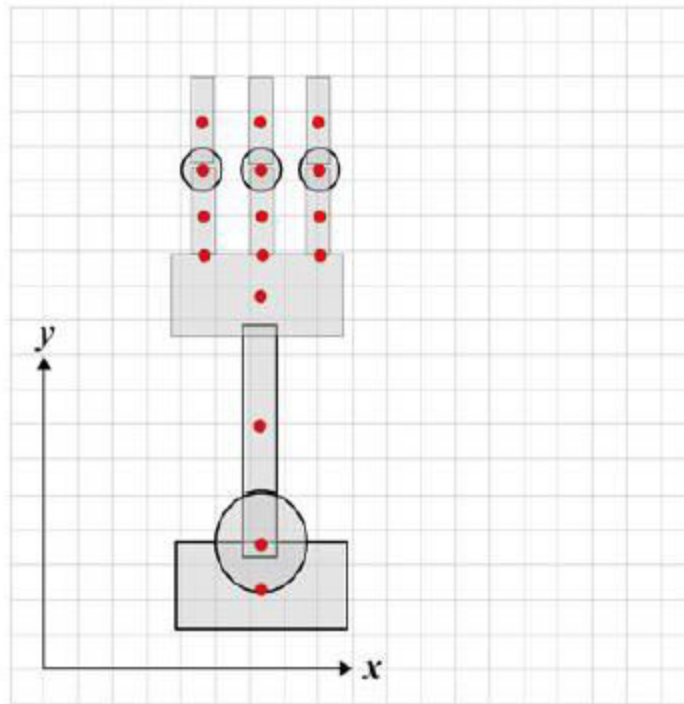
Hierarchical Transformations

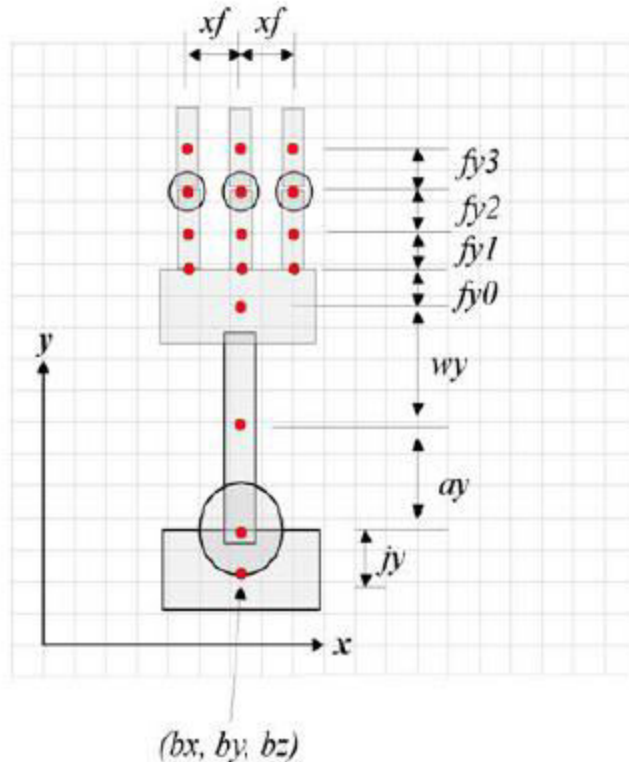


Each finger is a child of the parent (wrist)
⇒ independent control over the orientation of the fingers relative to the wrist



Hierarchical Transformations





```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(bx, by, bz);
    create_base();
glTranslatef(0, jy, 0);
glRotatef(joint1_orientation);
    create_joint1();
glTranslatef(0, ay, 0);
    create_upperarm();
glTranslatef(0, wy);
glRotatef(wrist_orientation);
    create_wrist();
glPushMatrix(); // save frame
    glTranslatef(-xf, fy0, 0);
    glRotatef(lowerfinger1_orientation);
    glTranslatef(0, fy1, 0);
        create_lowerfinger1();
    glTranslatef(0, fy2, 0);
    glRotatef(upperfinger1_orientation);
        create_fingerjoint1();
    glTranslatef(0, fy3, 0);
        create_upperfinger1();
glPopMatrix(); // restore frame
glPushMatrix();
    // do finger 2...
glPopMatrix();
    // do finger 3...
glPopMatrix();
```

Finger1


```

void DrawRobot(void)
{
    /*-----BUST AND HEAD-----*/
    glPushMatrix();
    glRotatef(bust_angle_y,0,1,0);
    glRotatef(bust_angle_x,1,0,0);
    glPopMatrix();
    glCallList(bust);
    glCallList(head);
    glPopMatrix();

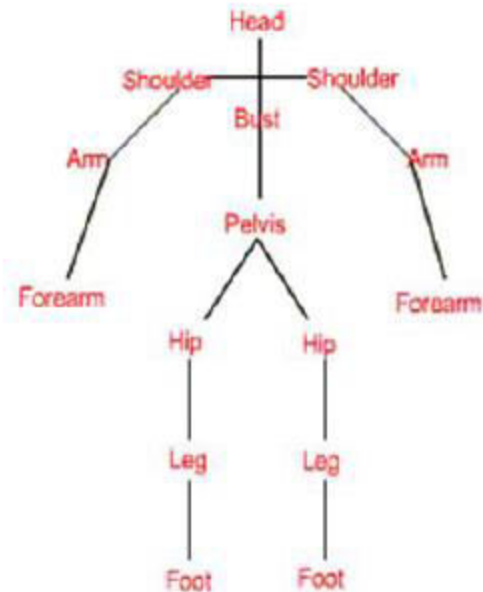
    /*-----RIGHT ARM AND FOREARM-----*/
    glPushMatrix();
    glTranslatef(0,0.25,0);
    glRotatef(right_arm_angle,1,0,0);
    glTranslatef(0,-0.25,0);
    glPopMatrix();
    glCallList(right_arm);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(1.25,-0.7,0);
    glRotatef(right_forearm_angle,1,0,0);
    glTranslatef(-1.25,0.7,0);
    glCallList(right_forearm);
    glPopMatrix();
    glPopMatrix();

    /*-----LEFT ARM AND FOREARM-----*/
    glPushMatrix();
    glTranslatef(0,0.25,0);
    glRotatef(left_arm_angle,1,0,0);
    glTranslatef(0,-0.25,0);
    glPopMatrix();
    glCallList(left_arm);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(1.25,-0.7,0);
    glRotatef(left_forearm_angle,1,0,0);
    glTranslatef(-1.25,0.7,0);
    glCallList(left_forearm);
    glPopMatrix();
    glPopMatrix();
}

```



'1' & 'Q'
 '2' & 'W'
 '3' & 'E'
 '4' & 'R'
 '5' & 'T'
 '6' & 'Y'
 '7' & 'U'
 '8' & 'I'

KEY ARROWS
 PGUP & PGDOWN
 'V' 'B' 'N' 'M'

right arm control
 right forearm control
 left arm control
 left forearm control
 left thigh control
 left leg control
 right thigh control
 right thigh control
 rotate model
 light control
 camera control



